



常州工学院
CHANGZHOU INSTITUTE OF TECHNOLOGY

课程大作业说明书

大数据与云计算（Q）

CulCloud 云盘与大数据分析平台

二级学院（直属学部）：计算机信息工程学院

专业：软件工程 班级：23 软件一

学生姓名：曹磊 学号：23030301

学生姓名：陈佳明 学号：23030302

学生姓名：陈晓楠 学号：23030303

学生姓名：程志阳 学号：23030304

学生姓名： 学号：

2026 年 6 月

摘 要

本文设计并实现了一个面向办公与教学资料管理场景的 CulCloud 云盘与大数据分析平台。在企事业单位日常办公、教师资料分发和学生实验归档过程中，用户经常面临文档散落在本地磁盘、网盘、格式转换工具和 PDF 批注工具之间的割裂问题，管理者也难以追踪文件处理链路中的真实任务状态。为此，系统围绕办公白领、教师和学生的高频文件处理需求与管理端监控诉求，集成了个人首页、云盘分类、格式转换、PDF Studio 和 Spark 数据驾驶舱等模块。系统提供可靠的文件预览、快速格式转换与在线 PDF 编辑功能，同时利用轻量级日志采集机制，为后端的离线数据分析平台提供全面而真实的操作轨迹数据。前端采用 React 与 TypeScript 构建交互界面，后端通过 FastAPI 提供服务，存储层结合 MinIO 与 HDFS，分析层使用 Spark 对遥测行为进行聚合分析。

针对原型中的功能缺口，本文完成了真实指标接入、虚拟目录树前缀算法、Gotenberg 异步转换队列（基于 Celery 与 Redis）以及 PDF canvas 双层批注渲染等关键实现的改造。系统支持画笔、图形、高亮、页面整理和重排导出，覆盖了办公报告批阅、教学资料整理、实验文档归档和多格式材料流转等主要场景。通过 Playwright 端到端测试和单元测试套件的全面验证，系统在高并发转换任务下保持了高成功率，且遥测大屏实现了秒级响应，验证了系统具备良好的高负载稳定性和低响应延迟。本项目的实施不仅提升了多类型文档的处理效率，也在开发中将分布式存储、异步队列和大数据计算等核心理论转化为工程实践，具有明确的工程训练与应用示范价值。

关键词：云盘；文件转换；PDF 编辑；Spark；数据可视化；办公协同

Abstract

This paper designs and implements CulCloud, an integrated cloud file management and big data analytics platform tailored for college course projects. Addressing the pain points of fragmented document workflows and simulated metrics in course demonstrations, the system provides an efficient student-oriented workspace and a telemetry-driven administrative cockpit. The frontend is constructed using React and TypeScript, while the backend employs FastAPI to expose file, preview, conversion, and task management APIs. Storage is managed with MinIO using virtual directories, with Gotenberg and a Celery-Redis queue handling asynchronous document transformations. For data analysis, Apache Spark processes user activity logs to render real-time visualizations.

To resolve initial design flaws such as static dashboard stats, flat file list layouts, cramped preview modals, and basic annotation support, we implemented key features: live operational metrics, drag-and-drop file organization, dedicated multiformat previews, and a canvas-based PDF Studio. The platform supports advanced drawing, highlighting, shape overlays, page organization, and PDF rebuilding. Comprehensive evaluation with Playwright e2e testing and pytest suites demonstrates stable processing under concurrent conversion loads and sub-second telemetry cockpit refreshes. CulCloud not only optimizes personal file-processing efficiency but also bridges theoretical cloud concepts with practical software engineering, demonstrating significant pedagogical value.

Keywords: Cloud Drive; Format Conversion; PDF Studio; Spark Telemetry; Data Visualization; Office Collaboration

目 录

第 1 章 绪论	1
1.1 研究背景与意义	1
1.1.1 政策驱动	1
1.1.2 市场与行业痛点	2
1.1.3 需求缺口	4
1.1.4 研究意义	4
1.2 国内外研究现状	5
1.2.1 云盘与文档管理产品现状	5
1.2.2 在线转换与 PDF 工具现状	5
1.2.3 研究评述	5
1.3 研究目的与主要内容	6
1.3.1 研究目的	6
1.3.2 主要内容	6
1.4 研究方法与技术路线	6
1.4.1 研究方法	6
1.4.2 技术路线	6
1.5 论文组织结构	7
第 2 章 相关软件及技术简介	8
2.1 前端开发技术	8
2.1.1 React 组件化开发	8
2.1.2 TypeScript 类型约束	8
2.1.3 Vite 与界面构建	9
2.1.4 HTML5 Canvas 交互渲染	9
2.2 后端服务技术	9
2.2.1 FastAPI 接口服务	9
2.2.2 Flask 分析服务	10
2.2.3 Redis 与 Celery 任务队列	10
2.2.4 MinIO 对象存储	11
2.3 文件处理技术	11
2.3.1 Office 与 PDF 转换	11

2.3.2	CSV、Markdown 与文本渲染	11
2.3.3	PDF 页面渲染与批注	12
2.4	运行与部署技术	12
2.4.1	PM2 进程管理	12
2.4.2	Docker Compose 容器编排	12
2.5	大数据分析技术	13
2.5.1	HDFS 与 Spark	13
2.5.2	ECharts 可视化	13
2.6	团队分工与协作机制	14
2.7	本章小结	15
第 3 章	系统分析	16
3.1	用户角色与用例分析	16
3.1.1	用户端角色	16
3.1.2	管理端角色	16
3.2	功能需求分析	17
3.2.1	核心功能需求	17
3.2.2	业务边界	18
3.3	非功能需求分析	19
3.4	数据需求分析	19
3.4.1	数据字典	19
3.4.2	数据模型与数据流	20
3.5	需求量化指标	21
3.6	可行性与合规边界	22
3.6.1	技术可行性	22
3.6.2	安全与合规边界	22
3.7	本章小结	22
第 4 章	系统总体设计	23
4.1	总体架构设计	23
4.2	系统功能设计	24
4.2.1	功能模块划分	24
4.2.2	用户端模块	25
4.2.3	管理端模块	25

4.3	业务流程设计	26
4.3.1	文件上传与预览流程	26
4.3.2	格式转换流程	27
4.3.3	PDF 编辑与导出流程	28
4.4	核心算法设计	29
4.4.1	预览类型分派算法	29
4.4.2	异步任务状态算法	30
4.4.3	PDF 页面导出算法	31
4.5	数据结构设计	31
4.5.1	MinIO 对象结构	31
4.5.2	Redis 状态结构	32
4.6	管理驾驶舱数据管线设计	32
4.7	部署结构设计	33
4.8	本章小结	34
第 5 章	系统详细设计及实现	36
5.1	个人首页模块的实现	36
5.1.1	真实指标聚合	36
5.1.2	失败原因回溯	37
5.2	我的文件模块的实现	37
5.2.1	上传、目录与类型分类	37
5.2.2	目录操作与对象移动	38
5.3	文件预览模块的实现	39
5.3.1	预览类型识别	39
5.3.2	结构化内容渲染	40
5.4	转换中心模块的实现	40
5.4.1	白名单与能力矩阵	40
5.4.2	异步任务执行	41
5.5	PDF 编辑室模块的实现	42
5.5.1	页面工作区	42
5.5.2	批注与导出	42
5.6	管理驾驶舱模块的实现	43
5.6.1	实时运行视图设计与实现	43
5.6.2	历史分析视图设计与实现	44

5.7 本章小结	46
第 6 章 系统测试	47
6.1 测试环境	47
6.1.1 软硬件环境	47
6.1.2 测试数据准备	47
6.2 测试层次与边界	48
6.3 功能测试	49
6.3.1 云盘与预览测试	49
6.3.2 转换链路测试	50
6.3.3 PDF 编辑测试	50
6.4 系统边界验证	51
6.5 问题处理与修复验证	54
6.5.1 文件移动路径边界	54
6.5.2 CSV 与 Markdown 预览边界	54
6.5.3 任务历史状态边界	54
6.5.4 PDF 导出文字与坐标边界	55
6.5.5 管理端联动边界	55
6.6 测试结果分析	56
6.6.1 核心业务闭环	56
6.6.2 稳定性与后续改进	56
6.7 本章小结	57
第 7 章 总结与展望	58
7.1 总结	58
7.1.1 系统功能成果	58
7.1.2 工程实践成果	58
7.2 不足之处	58
7.2.1 PDF 批注写回不足	58
7.2.2 Office 深度预览不足	59
7.2.3 权限与协作能力不足	59
7.3 展望	59
7.3.1 增强 PDF 编辑能力	59
7.3.2 完善多格式文件处理	59
7.3.3 扩展数据分析指标	59

致 谢	60
参考文献	61

第 1 章 绪论

本章围绕 CulCloud 云盘与大数据分析平台的研究起点展开，依次回答“为什么需要建设此类系统、现有产品和技术已经做到什么程度、本文准备从哪些方面完成设计与验证”三个问题。章节首先从政策要求、市场增长和办公教学资料管理痛点说明研究背景与现实意义，然后梳理云盘、在线转换和 PDF 工具的国内外发展现状，在此基础上明确系统研究目的、主要内容、研究方法、技术路线以及全文结构安排。

1.1 研究背景与意义

1.1.1 政策驱动

随着数字化办公与科研协作的普及，文档已从单一的本地物理文件演变为贯穿生命周期的连续数据对象。国家在网络安全、数据安全与个人信息保护等法律法规中指出，系统处理用户数据时应具备边界清晰、过程可追溯和责任可说明等能力 [1-3]。因此，云端资料平台必须在目录权限、异步转换、日志留存等环节融入安全合规设计。

《促进大数据发展行动纲要》将数据资源上升为国家战略资源，并强调释放数据价值 [4]。如图 1.1 所示，国家战略对大数据应用在经济转型与管理升级层面的重要价值提供了明确依据。对于 CulCloud 平台而言，这要求系统打破静态存储的局限，将上传、预览、转换等用户行为日志沉淀为可分析的数据链路，为后端的 Spark 聚合分析与管理驾驶舱提供真实的数据源。



索引号: 000014349/2015-00145

主题分类: 工业、交通\信息产业(含电信)

发文机关: 国务院

成文日期: 2015年08月31日

标题: 国务院关于印发促进大数据发展行动纲要的通知

发文字号: 国发〔2015〕50号

发布日期: 2015年09月05日

国务院关于印发促进大数据发展 行动纲要的通知

国发〔2015〕50号

各省、自治区、直辖市人民政府,国务院各部委、各直属机构:

现将《促进大数据发展行动纲要》印发给你们,请认真贯彻落实。

国务院

2015年8月31日

(本文有删减)

一、发展形势和重要意义

全球范围内,运用大数据推动经济发展、完善社会治理、提升政府服务和监管能力正成为趋势,有关发达国家相继制定实施大数据战略性文件,大力推动大数据发展和应用。目前,我国互联网、移动互联网用户规模居全球第一,拥有丰富的数据资源和应用市场优势,大数据部分关键技术研发取得突破,涌现出一批互联网创新企业和创新应用,一些地方政府已启动大数据相关工作。坚持创新驱动发展,加快大数据部署,深化大数据应用,已成为稳增长、促改革、调结构、惠民生和推动政府治理能力现代化的内在需要和必然选择。

(一) **大数据成为推动经济转型发展的新动力。**以数据流引领技术流、物质流、资金流、人才流,将深刻影响社会分工协作的组织模式,促进生产组织方式的集约和创新。大数据推动社会生产要素的网络化共享、集约化整合、协作化开发和高效化利用,改变了传统的生产方式和经济运行机制,可显著提升经济运行水平和效率。大数据持续激发商业模式创新,不断催生新业态,已成为互联网等新兴领域促进业务创新增值、提升企业核心价值的重要驱动力。大数据产业正在成为新的经济增长点,将对未来信息产业格局产生重要影响。

(二) **大数据成为重塑国家竞争优势的新机遇。**在全球信息化快速发展的大背景下,大数据已成为国家重要的基础性战略资源,正引领新一轮科技创新。充分利用我国的数据规模优势,实现数据规模、质量和应用水平同步提升,发掘和释放数据资源的潜在价值,有利于更好发挥数据资源的战略作用,增强网络空间数据主权保护能力,维护国家安全,有效提升国家竞争力。

图 1.1 《促进大数据发展行动纲要》来源信息与关键政策表述

1.1.2 市场与行业痛点

近年来,全球非结构化数据规模急剧增长。根据 Fortune Business Insights 的研究,全球云存储市场规模在 2025 年约为 1612.8 亿美元,预计 2034 年将增至 8099.9 亿美元 [5]。如图 1.2 所示,远程协作、大容量存储与自动化分析推动了该市场的扩张,说明高效的文件管理已成为数字化办公的核心基础设施。

云端资料管理需求的背景

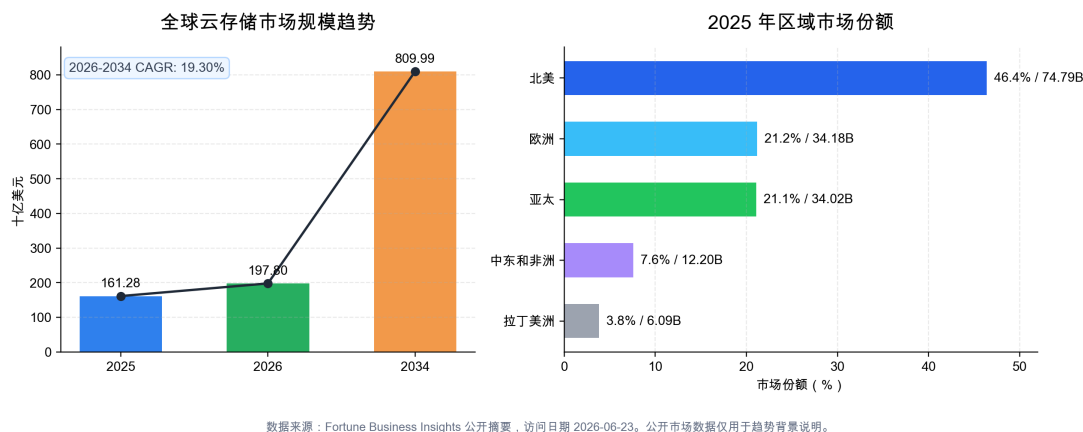


图 1.2 云端资料管理需求的背景

在云存储基础上，多格式在线转换的需求呈增长趋势。DataIntello 数据显示，全球文件转换软件市场在 2025 年达 28.0 亿美元，预计 2034 年将增长至 62.0 亿美元 [6]。如图 1.3 所示，异构数据的转换需求分布中，以 PDF 和 Office 为代表的文档转换占比达 19.8%，图像与音视频也占较大比重。这表明平台必须设计高可用的异步转换白名单机制，以保证文档标准化流转的响应效率。

多格式文件在线转换市场的规模与构成

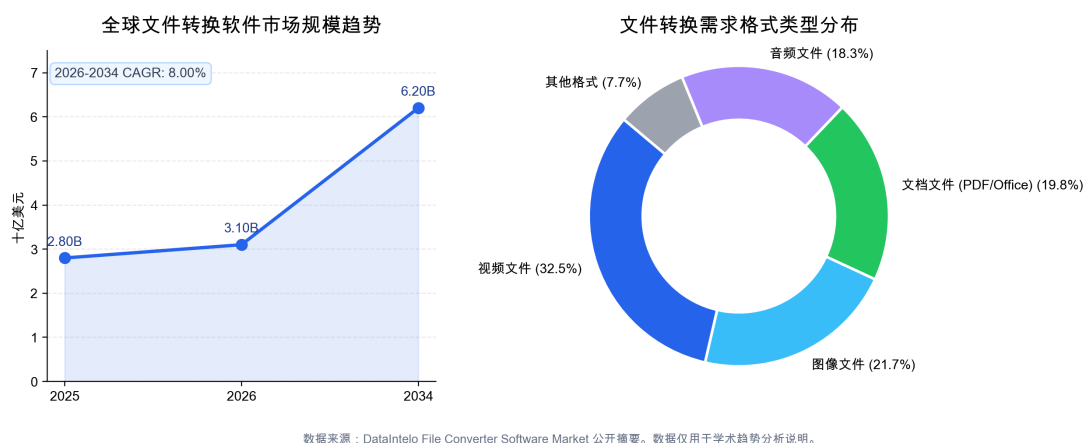


图 1.3 全球多格式文件转换软件市场趋势与需求分布

此外，PDF 文档的在线编辑与页面重组需求也越发迫切。DataIntello 指出，全球 PDF 编辑软件市场将在 2034 年达到 112.0 亿美元 [7]。如图 1.4 所示，PDF 因跨平台一致性被广泛采用。调研发现，在 Adobe Acrobat Online 等 5 款主流工具中，页面提取、旋转删除、文档合并等高频操作支持率达 100%，而复杂的表单设计等需求相对较少。这表明用户最核心的痛点在于“轻量级页面重排与基本批注”，通过

在资料库内集成 PDF Studio，可显著减少在第三方工具间跳转的文件流失和效率损耗。

全球 PDF 编辑软件市场规模与在线核心工具需求特征

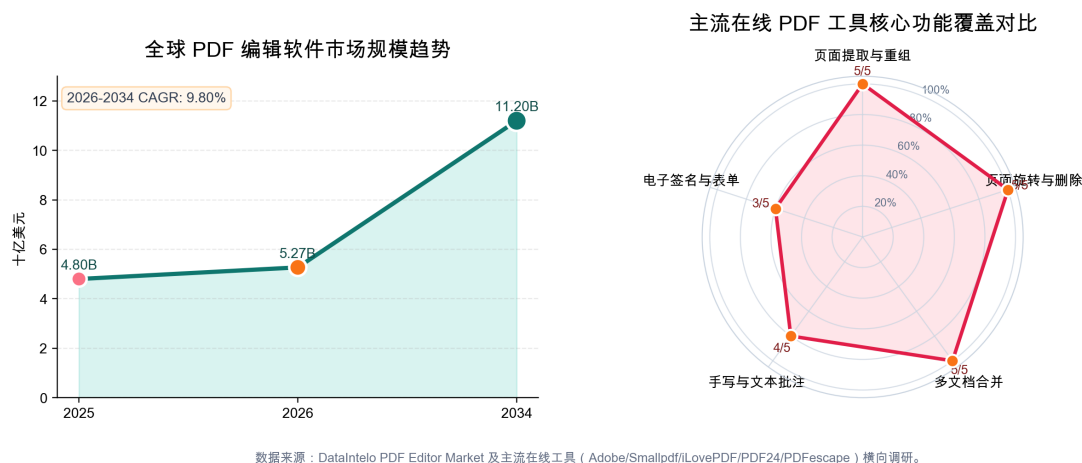


图 1.4 全球 PDF 编辑软件市场规模与主流工具功能覆盖率

在实际场景中，教师的课件和批注稿、学生的实验记录与报告草稿、办公人员的合同与报表，往往分散在本地和各类在线转换站中。文件在不同工具中迁移导致转换前后关联丢失。这就要求系统提供一个统一的资料工作台，实现文件在目录结构下的分类、预览、转换与批注的无缝衔接。

1.1.3 需求缺口

现有通用云盘具备成熟的同步、分享和基础预览能力，但通常不会围绕本地转换队列、PDF 批注写回、任务历史和后台遥测做深度整合。在线转换工具能够快速完成单次格式转换，却往往把转换前后的文件关系留给用户自行维护。单独的 PDF 工具具备较强编辑能力，但与资料库、文件夹路径和后台数据分析链路相对割裂。CulCloud 的研究切入点正是把这些分散能力收束到一个以“文件路径”为核心的闭环中，使文件进入系统之后能够继续被分类、预览、转换、批注、导出和分析。

1.1.4 研究意义

本研究的实践意义在于降低中小规模办公与教学资料管理的操作成本。系统通过统一上传入口、文件夹路径元数据、类型化预览、异步转换队列和 PDF Studio 批注能力，使用户不必在多个工具之间频繁切换。其工程意义在于验证前端交互、后端任务编排、对象存储、转换服务和日志分析之间如何形成清晰边界。其数据意

义在于把用户操作转化为可追溯、可聚合、可展示的行为记录，为后续容量规划、失败定位和功能改进提供依据。

1.2 国内外研究现状

1.2.1 云盘与文档管理产品现状

Google Drive、Dropbox 等国际云盘产品通常将拖拽上传、文件夹组织、类型图标、文件预览、共享链接和多端同步作为基础能力 [8-9]。这类产品的优势在于生态成熟、协作能力强和跨端体验稳定，但其功能重心更多放在云端同步与协同办公，而不是围绕单个文档从上传、转换、批注到日志分析的完整处理链路。国内同类产品也普遍重视容量、同步和分享体验，适合大规模资料存储，但对轻量部署、局部可控数据、转换队列可解释性和实验性分析大屏的支持通常较弱。

1.2.2 在线转换与 PDF 工具现状

Smallpdf、iLovePDF、Adobe Acrobat Online 等在线服务提供 PDF 合并拆分、Office 与 PDF 互转、图片处理等能力，能够满足用户临时处理文档的需求。但是这类服务通常以单次任务为中心，用户上传文件后获得转换结果，任务历史、目录归属、失败原因和二次预览并不会自然回到自己的资料库。另一方面，专业 PDF 编辑器在批注、页面整理和导出方面能力较强，却往往需要独立安装或付费订阅。对于办公和教学资料处理而言，更常见的需求并非复杂排版编辑，而是可靠预览、轻量批注、结果回收和低门槛使用。

1.2.3 研究评述

综合来看，现有产品已经在云端存储、在线转换和 PDF 编辑三个方向形成成熟能力，但这些能力之间仍存在明显断点。云盘解决“存储与查找”，转换工具解决“格式变化”，PDF 工具解决“阅读与批注”，而用户真正需要的是跨越三者的连续工作流。CulCloud 因此不追求替代大型云盘或专业编辑器，而是选择围绕高频资料处理场景构建轻量闭环：以文件夹路径承载文件组织，以转换任务承载格式处理，以 PDF Studio 承载在线批注，以日志与遥测承载可观察性和后续数据分析。

1.3 研究目的与主要内容

1.3.1 研究目的

本文旨在设计并实现一个面向办公白领、教师和学生的云端资料处理平台，使用户能够在统一界面中完成资料上传、目录整理、文件预览、格式转换、PDF 批注、结果下载和操作追踪。系统需要兼顾普通用户的易用性和管理端的可观察性：用户侧强调路径清晰、预览稳定、转换可恢复；管理侧强调任务状态可解释、失败记录可回溯、关键行为可聚合。

1.3.2 主要内容

围绕上述目标，本文完成了三个方面的工作。第一，设计“我的文件”模块，实现文件夹树、文件拖拽上传、路径元数据、类型分类、移动、删除、下载和预览等基础能力，使资料库具备接近真实云盘的组织能力。第二，构建格式转换与预览链路，基于后端白名单控制高频转换类型，并结合 **Gotenberg**、文本解析、**Markdown** 渲染、**CSV** 表格预览和图片/PDF 直接预览，降低用户处理异构文件的成本。第三，完善 **PDF Studio** 与管理驾驶舱，使用户能够完成页面预览、轻量批注、页面整理和导出，同时将上传、预览、转换和下载等行为沉淀为日志，为 **Spark** 聚合分析提供数据基础。

1.4 研究方法与技术路线

1.4.1 研究方法

本文主要采用文献与资料分析、竞品对比、原型实现和测试验证相结合的方法。文献与资料分析用于明确数据安全、大数据发展和云存储市场背景；竞品对比用于识别通用云盘、在线转换工具和 **PDF** 工具之间的功能边界；原型实现用于验证 **React**、**FastAPI**、**MinIO**、**Redis/Celery**、**Gotenberg** 和 **Spark** 等组件能否组成稳定链路；测试验证则通过白盒测试、端到端测试和日志审计检验文件移动、转换、预览、批注和持久化能力。

1.4.2 技术路线

系统技术路线遵循“文件进入、对象存储、任务编排、结果回收、行为分析”的顺序。前端通过 **React** 与 **TypeScript** 实现文件列表、目录树、转换中心、**PDF Studio** 和管理驾驶舱；后端通过 **FastAPI** 提供文件、目录、任务和日志接口；文件实体存入 **MinIO**，并在数据库中记录路径、类型和归属关系；转换任务进入 **Redis/Celery**

队列，由 **Gotenberg** 或本地解析逻辑生成预览结果；操作日志进入分析链路，最终由 **Spark** 聚合为可视化指标。该路线既能覆盖普通用户的连续操作，也能满足系统运行状态的可观察性要求。

1.5 论文组织结构

本文共分为七个章节。第一章为绪论，阐述研究背景、市场需求、现有产品状况、研究目标、主要内容和技術路线。第二章为相关技术介绍，说明 **React**、**TypeScript**、**FastAPI**、**MinIO**、**Redis/Celery**、**Gotenberg**、**PDF.js**、**HDFS** 与 **Spark** 在系统中的作用。第三章为系统分析，从用户角色、功能需求、非功能需求和可行性角度明确系统边界。第四章为总体设计，给出系统架构、部署方式、数据结构和任务流转设计。第五章为详细设计与实现，展开云盘管理、格式转换、**PDF Studio** 和管理驾驶舱的核心实现。第六章为系统测试，说明功能测试、转换矩阵测试、预览测试、持久化验证和日志审计结果。第七章为总结与展望，概括已完成成果，并讨论 **Office** 预览、批注写回和多媒体资料处理等后续优化方向。

第 2 章 相关软件及技术简介

本章介绍 CulCloud 平台开发过程中使用的主要技术，包括前端框架、后端服务、对象存储、任务队列、文件转换、PDF 渲染和大数据分析组件。

2.1 前端开发技术

2.1.1 React 组件化开发

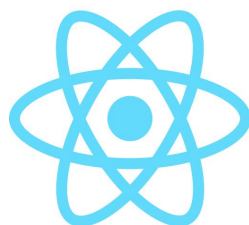


图 2.1 React 标识

前端采用 React 框架构建模块化的单页面应用 [10]。系统将 Dashboard、My-Files、ConvertCenter、PDFStudio 等业务视图抽象为独立组件，并设计了全局 DashboardContext 机制（见 file-cloud-frontend/src/context/useDashboard.tsx），统一维护视图流转与用户角色。在角色切换时，系统触发组件树重渲染，并在顶层容器动态绑定 theme-admin 样式，实现界面主题与视图权限的平滑过渡。

2.1.2 TypeScript 类型约束



图 2.2 TypeScript 标识

系统引入 TypeScript 进行静态类型约束，所有通信接口与状态协议均在 file-cloud-frontend/src/types.ts 中定义。通过对 FileItem、Task 等核心数据结构进行显式建模，在编译期捕获类型不匹配与属性缺失问题，保障文件预览、异步转换等多分支业务流转时的运行时稳定性。

2.1.3 Vite 与界面构建



图 2.3 Vite 标识

Vite 负责前端代码的开发调试与最终构建。利用其原生的 ESM 按需编译能力提升频繁交互界面（如 PDF 批注工作区）的响应开发效率。构建产物输出为优化后的静态资源，交由服务端托管部署。

2.1.4 HTML5 Canvas 交互渲染



图 2.4 HTML5 Canvas 标识

HTML5 Canvas 是 PDF 工作区与管理驾驶舱的可视化基础 [11]。在 PDF 批注中，系统在 Canvas 层上记录用户画笔、矩形和文本的相对坐标，并在导出时传给后端换算。同时，管理端 ECharts 默认采用 CanvasRenderer 绘制拓扑与统计图表，提供高效的可视化呈现。

2.2 后端服务技术

2.2.1 FastAPI 接口服务

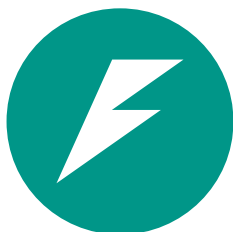


图 2.5 FastAPI 标识

后端基于 FastAPI 构建高效的 REST API 路由 [12]。利用 Pydantic 结构化校验前端提交的各类请求（如 PDF 页面重组请求，详见 `backend/app/models/schemas.py`），确保路由入口层的数据安全性；配合依赖注入机制，解耦鉴权、MinIO 客户端与任务状态调度逻辑。

2.2.2 Flask 分析服务



图 2.6 Flask 标识

Flask 框架用于承载独立的分析数据接口 [13]。该分析服务与 FastAPI 解耦，专门用于读取 Spark 分布式聚合产物及服务遥测数据，并将其映射为管理驾驶舱所消费的历史指标接口，从而实现主业务流与分析遥测流在架构上的分层。

2.2.3 Redis 与 Celery 任务队列



图 2.7 Redis 与 Celery 标识

Redis 与 Celery 协同实现文件转换的异步任务队列。Redis 作为消息 Broker 并缓存近期任务的运行状态（含 `queued`, `processing`, `success`, `failed`，详见 `backend/app/services/task_queue.py`）。通过将计算密集型的文档转换（如 LibreOffice 转换）从同步 HTTP 请求中解耦，前端可发起状态轮询展示进度，保证了核心接口的响应性能。

2.2.4 MinIO 对象存储



图 2.8 MinIO 标识

系统采用 MinIO 充当本地对象湖存储 [14]。后端通过“对象名路径前缀”（详见 `backend/app/routers/folders.py`）模拟分层文件夹，重命名或移动时，级联拷贝前缀匹配的所有对象再批量清除源对象，在对象存储扁平架构上提供了高效的层级文件目录视图。

2.3 文件处理技术

2.3.1 Office 与 PDF 转换

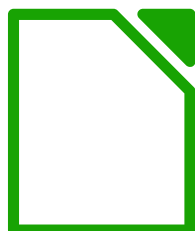


图 2.9 LibreOffice 与 Gutenberg 标识

系统集成 Gutenberg 转换服务处理异构文档格式 [15]。该服务容器化集成了 LibreOffice 和 PDF 引擎，后端通过 HTTP 接口提交 Office 转 PDF 任务，避免了直接在主业务进程中拉起本地进程的开销，结合 P0 格式白名单确保转换队列的可靠运行。

2.3.2 CSV、Markdown 与文本渲染

对于 CSV、Markdown 和 TXT 等轻量文本，系统采用本地轻量解析技术。后端直接提取文本并将其渲染为隔离的 HTML，前端通过预览容器直接呈现，无需触发繁重的文档转换流程。

2.3.3 PDF 页面渲染与批注



图 2.10 PDF.js 标识

对于 PDF 页面渲染与管理，系统以 PDF.js 的逻辑视口机制作为设计参照 [16]。PDFStudio 采用双层画布模型：底层展示后端 PyMuPDF 导出的高清页面图像，上层在 HTML5 Canvas 叠加手写及文本批注。前端记录相对视口的逻辑坐标与分辨率，导出时由后端进行坐标变换并合并写入新文件。

2.4 运行与部署技术

2.4.1 PM2 进程管理

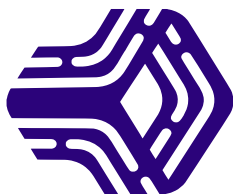


图 2.11 PM2 标识

PM2 用于管理应用层的前端与分析模块服务 [17]。根目录下的 `ecosystem.config.js` 统一配置了开发与生产的服务进程。PM2 支持异常退出自动拉起与集中日志查阅，保障本地开发和现场验证时的服务连续性。

2.4.2 Docker Compose 容器编排



图 2.12 Docker Compose 标识

Docker Compose 用于编排系统底层的 Redis、MinIO、Gotenberg 及 API 镜像 [18]。通过将持久化存储挂载、网络端口与多服务拓扑写入 docker-compose.yml 声明式配置，系统形成了“PM2 监管上层应用、Compose 监管基础设施”的混合部署架构。

2.5 大数据分析技术

2.5.1 HDFS 与 Spark



图 2.13 Hadoop 与 Spark 标识

HDFS 与 Spark 构成了本平台的大数据离线遥测分析管线 [19-20]。遥测流水日志进入存储后，Spark 计算引擎执行周期性的数据清洗、操作统计、错误率计算和格式流转矩阵分析，聚合出的指标存入静态 JSON，打通了“业务日志-数据清洗-指标分析-结果应用”的数据流。

2.5.2 ECharts 可视化



图 2.14 Apache ECharts 标识

前端集成 ECharts 图表库进行多维度数据呈现。通过对实时事件流与 Spark 历史数据进行提取，使用折线图展示流量趋势、柱状图展示格式分布、雷达图展示质量监控、Sankey 拓扑图展示格式流转，提升了管理端驾驶舱的可视化运营密度。

2.6 团队分工与协作机制

为确保 CulCloud 云盘与大数据分析平台各项复杂技术的顺利落地与系统级集成，项目团队制定了高度工程化的团队分工与协作机制。各小组成员具体分工如下：

曹磊（组长）：全面负责系统的基础架构设计与核心后端开发。设计并实现了基于 FastAPI 的 REST API 接口体系、MinIO 级联目录与对象湖迁移逻辑、Redis 与 Celery 异步任务队列调度，以及 Gonenberg 文档转换引擎的容器化集成；同时，主导了基于 Apache Spark 的分布式离线分析管线，设计了管理驾驶舱的实时拓扑图与运行日志流面板，并编写了覆盖系统核心交互链路的 11 项 E2E 自动化测试用例，为代码集成合并提供了严格的质量保障。

陈佳明（组员）：负责用户端基础页面与文件管理前端交互开发。实现了“个人首页”与“我的文件”模块的前端渲染，包括多维度分类过滤器、路径面包屑导航、文件拖拽上传列表等 React 组件编写，并完成了全站的 CSS 样式适配与响应式布局调整，实现了用户端界面的平滑流转。

陈晓楠（组员）：负责 PDF Studio 处理面板前端视图开发。实现了 PDF 卡片拖拽重排与旋转/删除的状态管理，编写了 Canvas 轻量化手写及文本批注交互逻辑，并与组长协作调试了前端视口逻辑坐标到后端物理 PDF 页面坐标的换算与批注写回逻辑。

程志阳（组员）：负责格式转换控制台与分析接口辅助开发。实现了“转换中心”前端面板，编写了格式转换任务提交与基于定时器的状态轮询终止逻辑，同时协助组长开发了 Flask 离线分析微服务中的指标读取与遥测数据查询接口。

为了在系统验收与评审中展现系统工程的严密性，防止因代码细节引发追问，团队制定了高度工程化的协作与版本管理策略。首先，团队采用 Schema-First（接口先行）协作模式，开发初期通过 FastAPI 的 OpenAPI 文档，基于 Pydantic 明确定义了所有 API 请求与响应的 JSON 数据格式（Schema），形成了前后端的强契约关系，允许前端组员使用 Mock 数据并行开发视图，而组长在后端同步实现真实业务逻辑。其次，在版本管理上采用 Git Flow 分支管理规范，由组长曹磊维护主分支 v2-platform，组员分别在各自的特性分支（如 feat-myfiles、feat-pdfstudio）上独立进行提交，互不干扰。最终在分支合并时，组员必须在本地运行通过组长编写的 Playwright 自动化测试集，所有用例通过后方可由组长合并至主分支，通过严密的质量网关保障了代码的稳定集成。

2.7 本章小结

本章介绍了系统涉及的前端、后端、对象存储、任务队列、文件转换、PDF 渲染、运行部署和大数据分析技术。这些技术共同支撑 CulCloud 从单纯文件列表扩展为可上传、可预览、可转换、可编辑和可分析的资料处理平台。

从工程分层看，React、TypeScript、Vite 和 Canvas 负责构建浏览器侧的高交互界面，FastAPI 负责承载文件处理主链路，Flask 则作为轻量分析服务承担驾驶舱数据接口，二者形成“主营业务服务”和“分析辅助服务”的边界。Redis 与 Celery 将耗时转换任务从同步请求中拆离，MinIO 负责对象化存储，Gotenberg、LibreOffice 和 PDF.js 分别补足 Office 转换、PDF 生成和页面渲染能力。PM2 与 Docker Compose 的组合则对应本地展示和容器化依赖两类运行场景，使系统既能快速预览，也能保持转换、缓存和对象存储等基础服务的可复现性。

这些技术并不是孤立堆叠，而是围绕文件流转闭环进行组合：用户上传资料后，后端写入对象存储并记录元数据；需要预览或转换时，任务队列调度转换服务生成结果；需要批注时，前端通过 PDF 页面图和 Canvas 批注层维护坐标关系；需要管理分析时，日志与遥测数据进入 Spark 聚合链路并由 ECharts 呈现。后续章节将在此基础上进一步说明需求建模、总体架构、核心模块实现和测试验证方法。

第 3 章 系统分析

本章在前文政策背景、市场调研和行业痛点的基础上，对 CulCloud 平台进行需求分析。分析对象限定为两个端侧：用户端与管理端。用户端面向普通文件处理者，负责资料上传、目录整理、文件预览、格式转换和 PDF 页面处理；管理端面向系统维护者，负责观察任务、日志、服务健康和历史分析结果。本章重点说明系统应完成什么、数据应如何被记录、哪些边界暂不纳入当前版本，为第 4 章系统总体设计提供依据。

3.1 用户角色与用例分析

3.1.1 用户端角色

用户端角色在代码中对应 `user`。该角色的主要目标是以云盘方式组织个人资料，并在同一平台内完成常见文件处理。用户端需要能够上传本地文件、创建文件夹、移动文件、按类型筛选、打开预览、提交格式转换任务、查看转换历史、进入 PDF 编辑室并导出处理后的 PDF 文件。用户端不直接访问 MinIO、Redis 或 Gotenberg，而是通过前端 API 客户端调用 FastAPI 接口，由后端完成对象写入、任务创建和结果返回。

3.1.2 管理端角色

管理端角色在代码中对应 `admin`。该角色不重复用户端的文件编辑操作，而是观察系统运行状态。管理端需要查看当前文件规模、任务成功率、失败任务、近期事件、队列长度、服务健康和历史分析样本。管理端第一屏必须使用 FastAPI 的实时接口聚合当前业务状态；Spark 与 ECharts 相关历史样本只能作为离线分析结果，不与实时指标混用。前端通过角色状态控制管理菜单可见性，后端敏感接口通过角色校验返回访问结果。

图 3.1 给出了系统用例关系。该图只保留用户端与管理端两个参与者，核心用例按文件管理、文件处理、PDF 编辑和运行观察组织，以减少图中交叉线条。用户端用例之间存在自然的包含关系，例如文件预览与格式转换均依赖文件选择；管理端运行观察包含任务统计、日志统计和服务健康读取。

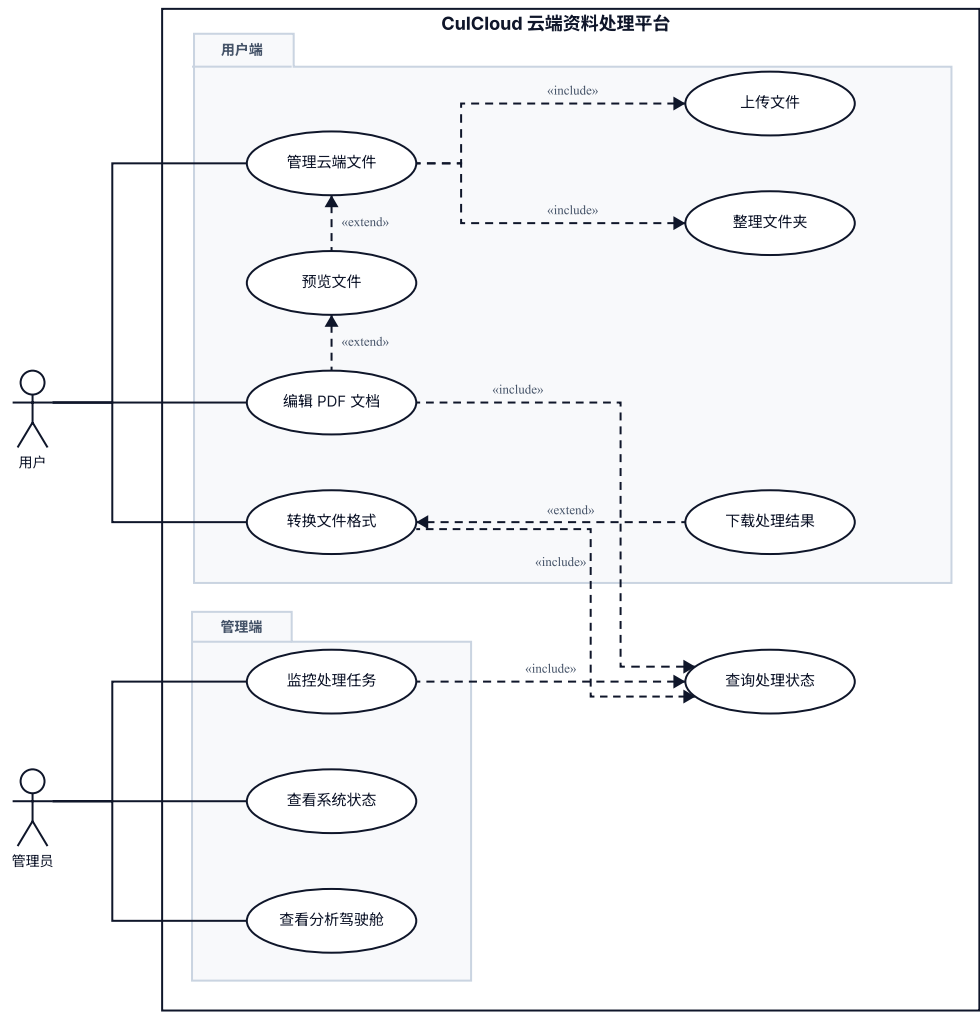


图 3.1 系统核心用例图

3.2 功能需求分析

3.2.1 核心功能需求

系统功能需求围绕“上传、预览、处理、观察”四条主线展开。上传要求文件能够进入对象存储并在列表中显示；预览要求不同类型文件能够返回合适的预览方式；处理要求格式转换和 PDF 页面导出能够生成可下载产物；观察要求用户端操作能够进入任务状态与事件日志，再被管理端读取。表 3.1 给出了功能需求矩阵。

表 3.1 功能需求矩阵

编号	模块	需求说明	验收依据
FR-01	文件管理	支持上传、列表、目录、移动、删除和下载	文件对象进入 MinIO，列表刷新后状态一致
FR-02	文件预览	支持 PDF、图片、文本、Mark-down、CSV、Office 等预览路径	预览接口返回类型与内容地址
FR-03	格式转换	支持 P0 白名单内格式转换与结果下载	任务进入终态并生成下载地址
FR-04	PDF 编辑	支持页面排序、旋转、删除、批注和导出	导出 PDF 可打开且保留页面配置
FR-05	任务与日志	记录上传、转换、导出、失败和健康事件	Redis 任务状态与日志接口可查询
FR-06	管理观察	展示实时运行指标与历史分析样本	实时接口与历史接口口径分离

3.2.2 业务边界

当前版本定位为最小可用的云端资料处理平台，重点保证单用户文件管理、轻量转换、PDF 页面处理和管理端运行观察。系统暂不实现多人协同编辑、跨设备冲突合并、企业级权限矩阵、文件版本回滚和长期审计归档。这些能力需要更复杂的关系数据模型、权限模型和审计策略，超出当前原型的主要验证范围。明确边界后，后续设计可以集中在文件对象、任务状态、运行日志和分析样本之间的一致性上。

从优先级看，文件上传、列表刷新、预览、格式转换、PDF 导出和任务状态查询属于 P0 需求，必须在系统主链路中稳定可用；目录整理、类型筛选、历史任务和管理端实时观察属于 P1 需求，用于提升系统完整度和可观察性；离线历史分析、质量热力图和历史转换桑基图属于 P2 需求，主要体现大数据分析能力。这样的优先级划分能够避免把历史分析样本误认为实时业务必需条件，也能保证核心文件处理链路优先完成。

从输入输出边界看，用户端输入主要包括本地文件、目标目录、目标格式、PDF 页面配置和批注对象；系统输出主要包括文件列表项、预览内容、任务编号、任务终态、结果下载地址和错误原因。管理端输入不是人工录入数据，而是读取系统运行过程中已经产生的文件、任务、日志和健康状态。该边界说明管理端图表的真实性依赖后端接口，而不是依赖前端写死的显示数值。

3.3 非功能需求分析

系统非功能需求包括可用性、稳定性、可维护性、安全性和可复现性。可用性要求页面具备加载、空状态、成功、失败和可恢复提示；稳定性要求长任务具有可查询状态，基础依赖异常时能够返回结构化错误；可维护性要求前端视图、API 客户端、后端路由和领域服务边界清晰；安全性要求不暴露对象存储内部地址，不采集与文件处理无关的敏感信息；可复现性要求 PM2、Docker Compose、接口探测和端到端用例可以重复执行。

对于可用性，系统不能只要求页面能够打开，还要要求操作链路能够闭合。例如上传完成后应能在文件列表中看到对象，转换完成后应能获得结果入口，PDF 导出完成后应能打开新文件。对于稳定性，系统需要把长任务的中间状态显式暴露给前端，避免用户无法判断任务是否仍在运行。对于可维护性，前端页面不能直接拼接对象存储内部地址，后端也不能把格式转换细节散落在多个路由中，而应通过领域服务统一处理。

对于安全性，系统的核心要求是最小暴露和路径清洗。文件名、目录前缀和结果地址都可能来自用户输入，后端必须统一规范化处理；下载与预览应通过 API 代理，避免把内部对象存储端口、桶名和临时对象路径直接暴露给浏览器。对于可复现性，测试不仅要覆盖页面操作，还要覆盖容器状态、进程状态、接口返回和自动化用例，使后续验证人员能够按固定顺序复查系统。

表 3.2 非功能需求矩阵

编号	类别	约束说明
NFR-01	可用性	关键操作应提供明确结果反馈，失败时返回可理解原因
NFR-02	稳定性	上传、转换和导出过程应记录任务状态，页面刷新后仍可查询
NFR-03	可维护性	前端、路由、服务和基础依赖之间通过清晰接口协作
NFR-04	安全性	文件路径必须清洗，内部对象地址不得直接暴露给浏览器
NFR-05	可复现性	服务状态、接口返回和端到端流程应能通过命令重复验证

3.4 数据需求分析

3.4.1 数据字典

系统当前没有引入复杂关系数据库，而是以 MinIO 对象、Redis 任务状态、Redis 事件日志和 Spark 历史样本组成数据层。表 3.3 给出了核心数据字典。该表用于说明后续 E-R 图和数据流图中的实体含义，避免把前端临时状态误认为持久数据。

表 3.3 核心数据字典

数据对象	关键字段	来源	用途
文件对象	objectName、size、contentType、updatedAt	MinIO	文件列表、下载、容量统计
目录对象	prefix、name、keepObject	MinIO 前缀	文件夹显示与移动边界
任务对象	taskId、kind、status、resultUrl、error	Redis	转换、PDF 导出和任务查询
事件日志	timestamp、eventType、actor、message	Redis	近期事件、日志统计和问题定位
健康快照	service、status、latency、checkedAt	FastAPI/Redis	系统状态与依赖可用性观察
历史样本	format、operation、count、quality	Spark JSON	输出 历史文件类型、转换趋势和质量分析

3.4.2 数据模型与数据流

图 3.2 给出了文件对象、任务对象和事件日志之间的概念关系。一个文件对象可以产生多个预览缓存、转换任务或 PDF 处理任务；一个任务最多生成一个结果产物；任务状态变化会产生事件日志。该关系用于约束后续设计：文件、任务、日志三者需要能够通过对象名或任务编号追溯，而不能只停留在前端按钮状态上。

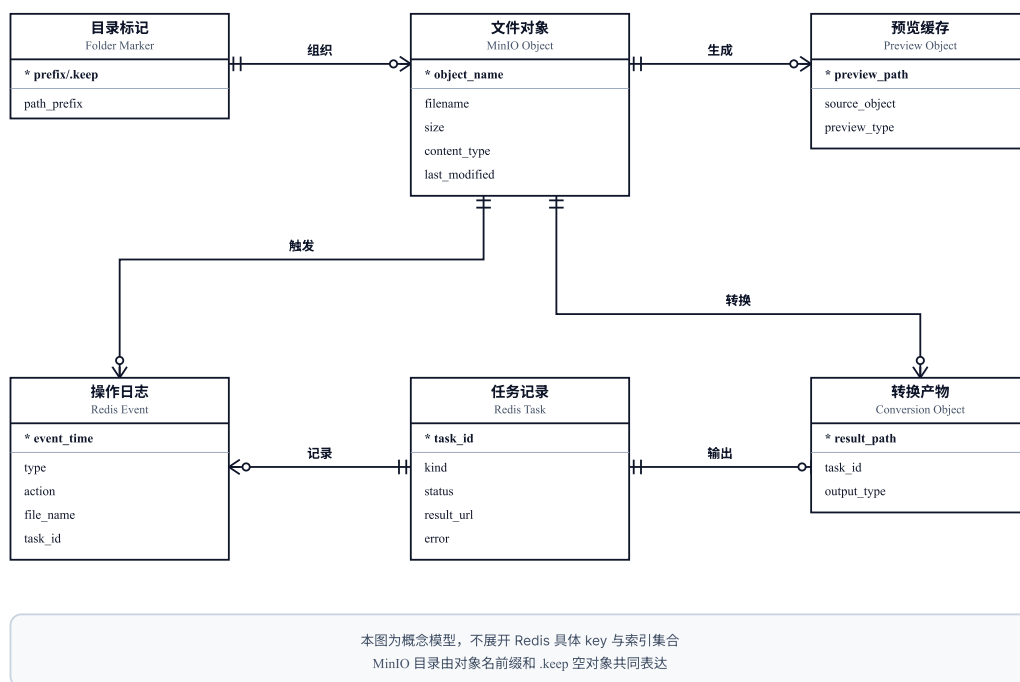


图 3.2 核心数据概念模型图

从数据流角度看，用户端操作首先进入 FastAPI 路由，路由调用领域服务写入 MinIO 或 Redis；当任务完成后，产物重新进入对象存储，状态和日志进入 Redis；管理端读取 FastAPI 聚合接口形成实时运行视图，历史分析页读取 Flask/Spark 结果形成离线分析视图。图 3.3 用需求层数据流表达了这一关系。

数据流分析还要求区分四类状态。第一类是浏览器临时状态，例如当前选中的文件、预览弹窗是否打开和 PDF 工作区缩放比例；第二类是对象存储状态，例如原始文件、预览缓存和转换产物；第三类是运行状态，例如任务进度、失败原因和近期事件；第四类是历史分析状态，例如 Spark 聚合后的文件类型分布和错误热力图。只有第二类和第三类能够直接反映当前业务结果，第四类只能用于历史分析，第一类则不应作为验收依据。

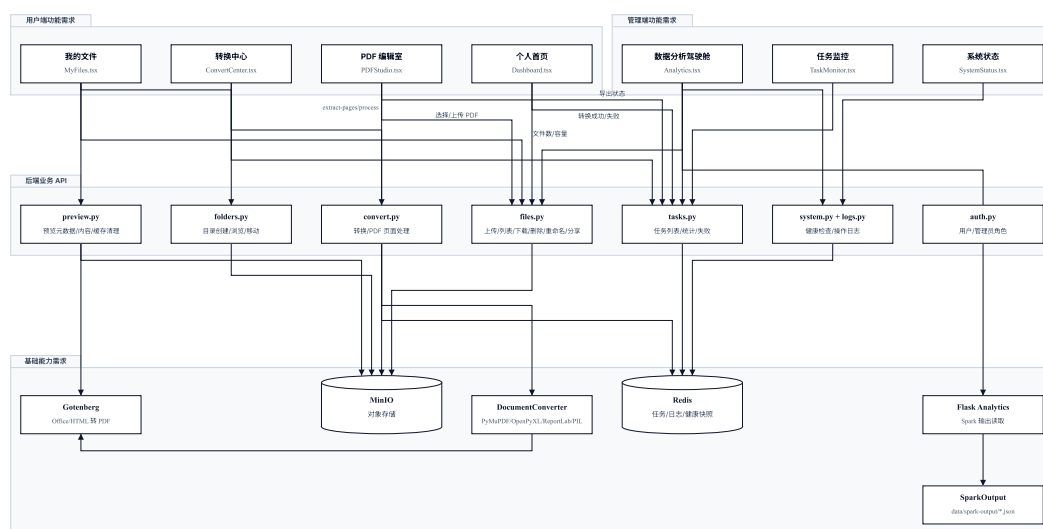


图 3.3 系统需求数据流图

3.5 需求量化指标

为便于测试和管理端观察，需求分析阶段需要定义若干可计算指标。设用户可见文件对象集合为 $O = \{o_1, o_2, \dots, o_n\}$ ，对象大小为 $size(o_i)$ ，则用户端容量需求指标为：

$$S = \sum_{i=1}^n size(o_i). \quad (3.1)$$

式 3.1 用于约束首页和管理端容量统计口径：目录占位对象、预览缓存和临时转换文件不应混入用户原始资料容量。

设等待、处理中、成功和失败任务数分别为 Q 、 P 、 C 、 F ，则实时任务总量为：

$$T = Q + P + C + F. \quad (3.2)$$

式 3.2 用于限定管理端第一屏的任务规模来自当前业务任务集合，而不是来自 Spark 历史样本。由此可以进一步得到成功率和失败占比，为第 4 章运行指标设计提供基础。

3.6 可行性与合规边界

3.6.1 技术可行性

React、FastAPI、MinIO、Redis、Gotenberg、Docker Compose、PM2、Spark 与 ECharts 均为成熟技术。项目代码已经具备前端视图、后端路由、对象存储、任务状态、预览服务、转换服务和分析展示基础，因此继续围绕主链路完善需求具有可行性。异步任务通过 Redis 状态与后端后台任务解耦，能够避免长时间转换阻塞普通接口；对象存储通过统一对象名前缀表达目录结构，能够满足当前文件管理需求。

3.6.2 安全与合规边界

系统只保存文件名、对象路径、大小、类型、任务状态和必要日志，不主动采集与文件处理无关的个人敏感信息。日志记录任务状态、操作类型和错误原因，但不记录文件正文内容。文件删除、移动和重命名均由用户主动触发，预览缓存与转换产物应与原文件隔离。若 Redis 或持久化卷不可用，近期日志和任务历史可能不完整，该边界需要在测试和系统状态中明确说明。

3.7 本章小结

本章从用户端与管理端两个角色出发，明确了功能需求、非功能需求、数据字典、概念数据模型、数据流和量化指标。需求分析的核心结论是：CulCloud 应围绕文件对象、任务状态、事件日志和分析样本建立可追溯链路。第 4 章将在此基础上进一步说明系统分层架构、业务流程、算法、公式、状态流转和部署结构。

第 4 章 系统总体设计

本章从总体架构、功能模块、业务流程、数据结构和部署结构五个方面说明 CulCloud 的系统设计。设计目标不是简单堆叠页面功能，而是让用户端文件处理、后端任务状态、对象存储、运行日志和管理端分析形成一致的数据闭环。系统采用 React 前端、FastAPI 业务服务、MinIO 对象存储、Redis 状态与日志、Gotenberg 转换服务以及 Flask/Spark 分析服务的组合结构，使文件上传、预览、转换、PDF 页面整理和管理端观察能够在同一平台内协同运行。

4.1 总体架构设计

CulCloud 按照表示层、业务接口层、领域服务层、基础数据层和分析应用层进行分层。表示层包含用户端与管理端两类界面，用户端负责文件上传、目录整理、预览、格式转换和 PDF 编辑，管理端负责任务、日志、健康状态和分析结果观察。业务接口层由 FastAPI 提供统一入口，按文件、目录、预览、转换、任务、日志和系统健康拆分路由。领域服务层封装对象存储、文档转换、任务追踪、日志采集和健康检查。基础数据层由 MinIO、Redis 和 Gotenberg 构成，分别承担对象保存、状态保存和文档渲染转换。历史分析层由 Spark 输出聚合 JSON，Flask Analytics 提供读取接口，再由管理端 ECharts 呈现。

图 4.1 给出了系统分层架构。该图强调两类数据口径：第一类是 FastAPI 直接聚合的实时运行数据，来自文件、任务、日志和系统健康接口；第二类是 Spark 处理后的历史分析样本，用于展示较长时间窗口下的文件类型、转换趋势和质量指标。二者在管理端同时出现，但语义上保持分离，避免将离线样本误认为实时业务总量。

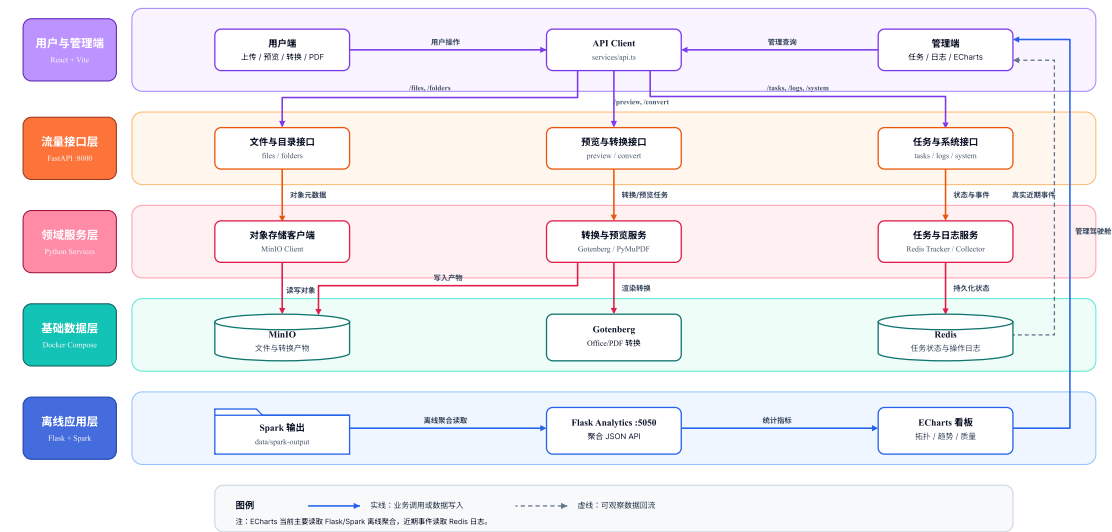


图 4.1 云端资料处理平台系统分层架构图

4.2 系统功能设计

4.2.1 功能模块划分

系统围绕用户端和管理端两条主线划分为六个核心模块。用户端包含个人首页、我的文件、转换中心和 PDF 编辑室；管理端包含管理驾驶舱、任务监控和系统状态。各模块并非孤立页面，而是通过统一 API 客户端访问后端接口，再由后端服务读写 MinIO 与 Redis。表 4.1 列出了模块边界与数据来源。

表 4.1 系统功能模块划分

端侧	模块	主要功能	数据来源
用户端	个人首页	文件数量、空间占用、近期任务和失败提示	文件接口、任务接口
用户端	我的文件	上传、目录、搜索、移动、删除和下载	MinIO 对象元数据
用户端	文件预览	PDF、图片、文本、Markdown、CSV、Office、音视频预览	预览接口、转换缓存
用户端	转换中心	P0 白名单格式转换、任务轮询、结果下载	转换接口、Redis 任务状态
用户端	PDF 编辑室	页面提取、排序、旋转、删除、批注和导出	PDF 处理接口、MinIO 产物
管理端	管理驾驶舱	实时运行状态、历史分析样本和图表展示	FastAPI、Flask/Spark、ECharts

图 4.2 展示了模块依赖关系。前端不直接访问 MinIO、Redis 或 Gotenberg，而是通过 API 客户端进入 FastAPI 路由，由路由层调用领域服务完成对象读写、转换执行和状态记录。这种设计将浏览器侧交互与基础设施细节隔离，降低了前端暴露存储地址或内部任务键的风险。

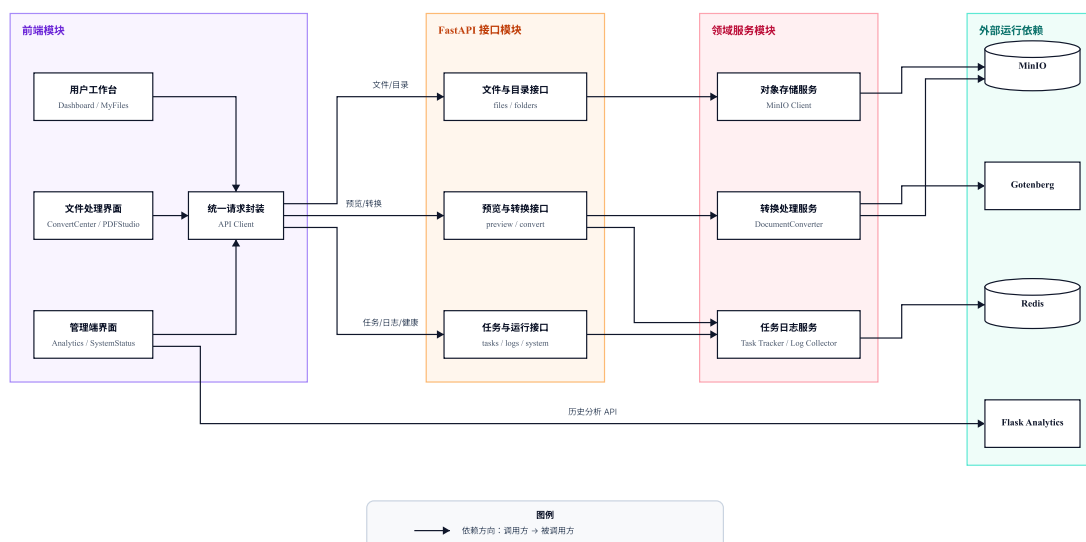


图 4.2 前后端核心模块依赖图

4.2.2 用户端模块

用户端的设计重点是让普通用户以云盘方式组织资料，并在同一界面路径中完成处理。我的文件模块负责对象列表、目录前缀、类型筛选和预览入口；转换中心负责从本地或云端选择文件，按白名单提交格式转换任务；PDF 编辑室负责将 PDF 页面转为可操作页面集合，再提交页面顺序、旋转角度和批注配置生成新文件。用户端所有可生成产物的操作都应写入任务状态和事件日志，以便后续查询与管理端观察。

4.2.3 管理端模块

管理端的设计重点是观察系统运行，而不是重复用户端操作。第一屏读取 FastAPI 的文件、任务、日志和健康接口，显示当前业务系统真实状态；第二屏和第三屏读取 Spark 历史分析样本，展示文件类型分布、历史转换任务、遥测质量、存储水位和错误热力图。任务监控用于查看任务列表、队列长度和失败原因；系统状态用于观察 Redis、MinIO、Gotenberg、API 和分析服务的健康状态。

4.3 业务流程设计

系统核心业务并不是单个页面动作，而是“用户操作、后端处理、对象产物、任务状态、管理观察”的闭环。用户上传、预览、转换和 PDF 导出会进入 FastAPI 路由，后端根据任务类型读写 MinIO 对象，并将任务状态与事件日志写入 Redis；管理端第一屏再从文件、任务、日志和健康接口读取实时数据。图 4.3 展示了这一业务闭环，它用于说明用户端业务增量为什么能够实时反映到管理端运行指标中。

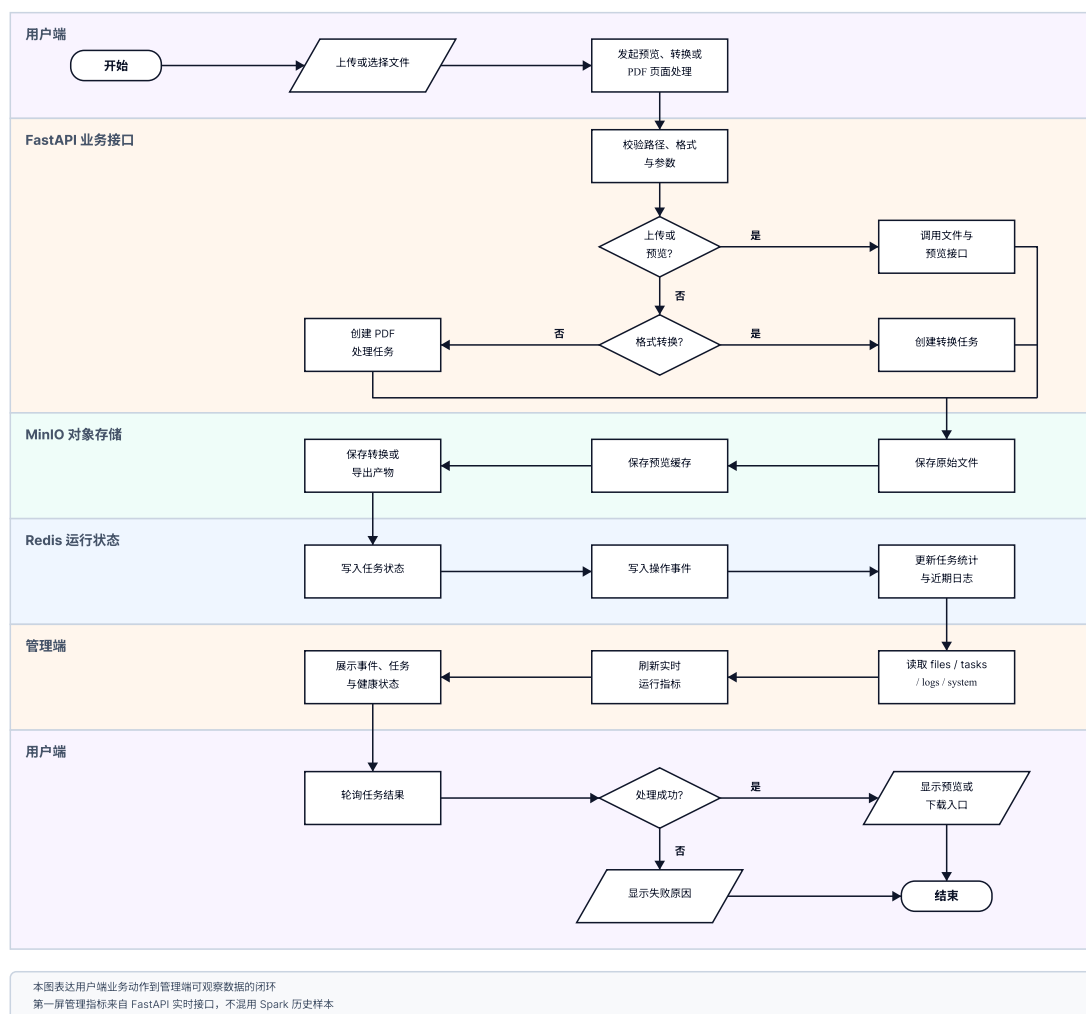


图 4.3 核心业务闭环流程图

4.3.1 文件上传与预览流程

文件上传流程从用户选择文件开始。前端携带当前目录前缀和相对路径调用上传接口，后端校验路径合法性后生成对象名称，将文件写入 MinIO，并记录上传事件。列表刷新时，后端根据对象名前缀和 `.keep` 空对象合成文件夹与文件视图。预览流程则由文件类型决定：图片、PDF、音频和视频可直接返回内容地址；TXT、Markdown 和 CSV 由后端生成可读内容；Office 文件优先生成 PDF 预览缓存。图

4.4 用活动图表示了上传、列表刷新和预览之间的责任边界。

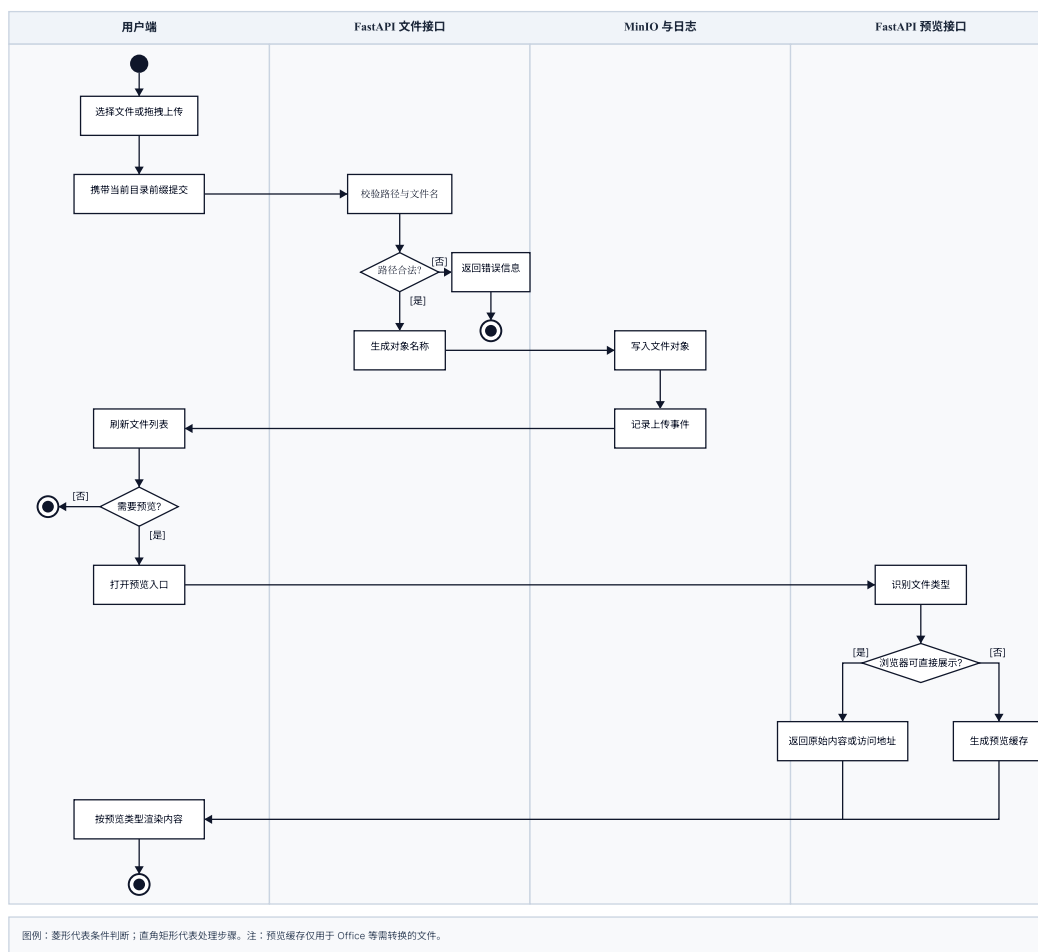


图 4.4 文件上传、整理与预览活动图

4.3.2 格式转换流程

格式转换采用“提交任务、后台处理、短轮询读取状态”的异步流程。用户在转换中心选择目标格式后，前端调用转换接口；后端首先执行 P0 白名单校验，通过后创建任务标识并写入 Redis，随后使用 FastAPI BackgroundTasks 启动后台转换逻辑。转换任务进入处理中状态后，从 MinIO 读取源文件，调用 Gotenberg 或本地转换库生成结果，最终将产物写回 conversions/ 前缀并更新任务状态。前端通过任务查询接口读取状态、错误信息和结果地址。图 4.5 展示了该时序。

该流程的设计重点有三点。第一，转换能力必须先经过白名单过滤，防止页面上未开放的实验能力被直接提交到后端执行。第二，转换结果统一通过后端文件内容接口代理下载，而不是把对象存储内部地址直接暴露给浏览器。第三，任务状态与日志事件同步写入，使“转换完成”“转换失败”“结果可下载”三个判断来自同一后端事实，而不是分别由前端按钮状态推断。

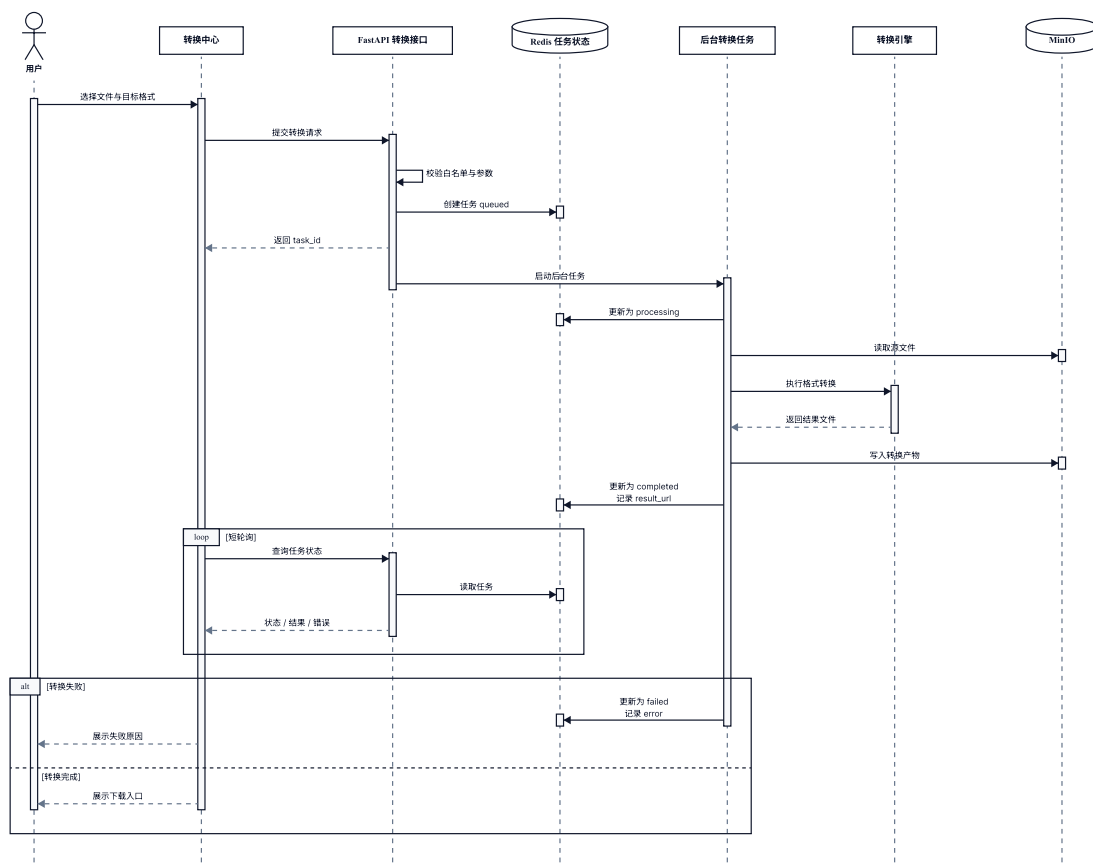


图 4.5 格式转换任务时序图

4.3.3 PDF 编辑与导出流程

PDF 编辑室将原始 PDF 拆解为页面图像，并在前端维护页面顺序、旋转角度、删除状态和批注对象。用户提交导出时，前端把页面配置传给 `/api/v1/convert/pdf/process`，后端根据配置重新组装 PDF 页面并写入批注，生成的新文件保存到 MinIO。该流程的关键设计是将“页面操作状态”与“最终 PDF 产物”分离：前端负责交互和坐标记录，后端负责文件级重组与输出。图 4.6 展示了 PDF 编辑与导出流程。

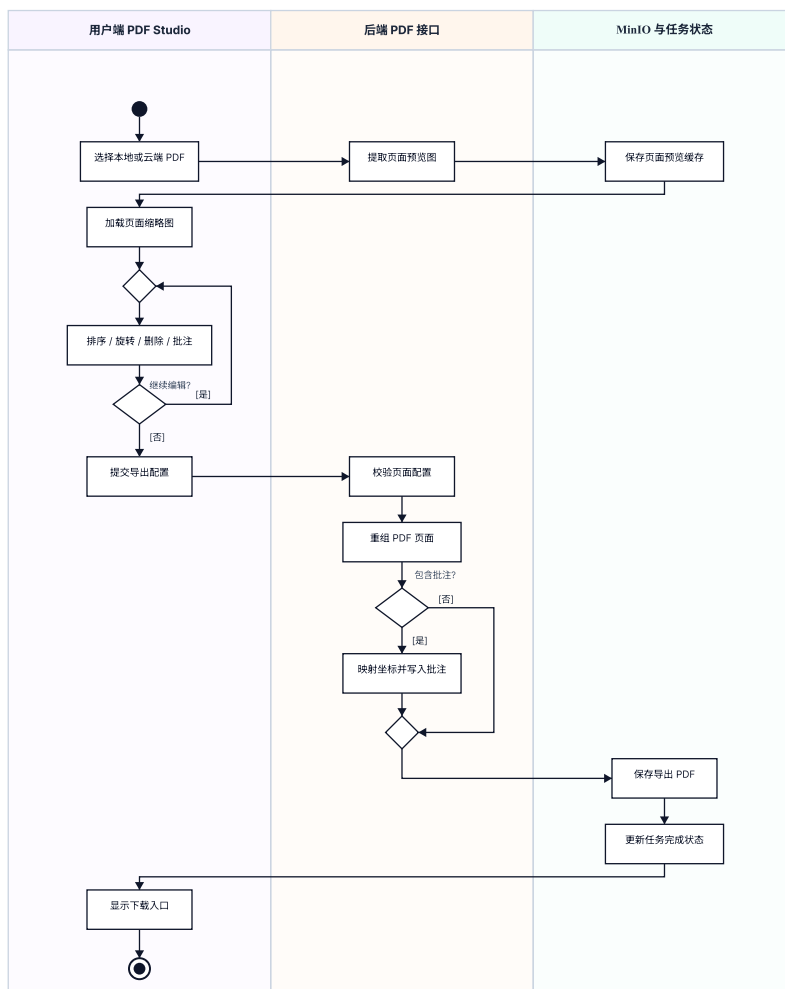


图 4.6 PDF 编辑室页面整理与导出活动图

4.4 核心算法设计

4.4.1 预览类型分派算法

文件预览需要在多种格式之间做稳定分派。设计上，前端不直接解析所有文件，而是由后端根据对象名和扩展名返回预览类型、内容地址和可预览状态。算法 4.1 给出了预览分派规则。该算法把二进制直读、文本结构化渲染、Office 预览缓存和不可预览状态区分开，便于第 5 章在 `preview.py` 与 `services/preview.py` 中落地。

算法 4.1 文件预览类型分派算法

输入： 文件对象名 *objectName*，文件扩展名 *ext*
 输出： 预览类型 *previewType* 与内容地址 *contentUrl*

- 1: **if** *ext* 属于 PDF、图片、音频或视频 **then**
- 2: 返回后端内容代理地址
- 3: **else if** *ext* 属于 TXT、Markdown 或 CSV **then**
- 4: 读取对象内容并生成结构化响应
- 5: **else if** *ext* 属于 Office 文档 **then**
- 6: 查询或生成 PDF 预览缓存
- 7: **else**
- 8: 返回不可预览状态和下载入口
- 9: **end if**

4.4.2 异步任务状态算法

格式转换和 PDF 导出均属于耗时文件处理任务。系统采用“先建任务、后处理文件、再更新终态”的设计，避免用户请求长时间阻塞。算法 4.2 给出了异步处理的统一状态更新规则。任务必须经历等待、处理中、成功或失败状态，成功路径需要写入对象产物和结果地址，失败路径需要写入错误原因。

算法 4.2 异步文件处理任务状态更新算法

输入： 任务编号 *taskId*，输入文件 *file*，处理类型 *kind*
 输出： 任务终态、结果地址或错误信息

- 1: 写入任务状态 *pending*
- 2: 后台任务启动，写入任务状态 *processing*
- 3: 记录开始事件到运行日志
- 4: **if** *kind* 为格式转换 **then**
- 5: 按白名单选择转换方法并生成结果字节
- 6: **else**
- 7: 按页面配置重组 PDF 页面与批注
- 8: **end if**
- 9: **if** 处理成功 **then**
- 10: 将结果写入 MinIO 的 *conversions/* 前缀
- 11: 写入任务状态 *success* 与 API 代理结果地址
- 12: 记录完成事件到运行日志
- 13: **else**
- 14: 写入任务状态 *failed* 与错误原因
- 15: 记录失败事件到运行日志
- 16: **end if**

前端轮询以任务终态作为停止边界。设第 k 次查询到的任务状态为 s_k ，终态集合为 $\Omega = \{success, failed\}$ ，则轮询停止条件为：

$$s_k \in \Omega. \quad (4.1)$$

式 4.1 可避免任务已完成后仍持续请求后端。

4.4.3 PDF 页面导出算法

PDF 编辑室的导出需要把前端页面顺序、旋转角度、删除状态和批注对象转化为后端可写入的 PDF 产物。批注导出的关键是坐标映射。设前端画布坐标为 (x_c, y_c) ，画布宽高为 W_c, H_c ，PDF 页面宽高为 W_p, H_p ，则写回 PDF 时的坐标可表示为：

$$x_p = x_c \cdot \frac{W_p}{W_c}, \quad y_p = y_c \cdot \frac{H_p}{H_c}. \quad (4.2)$$

式 4.2 说明批注位置应由画布尺寸与 PDF 页面尺寸之间的比例确定，而不是依赖浏览器缩放值。

算法 4.3 PDF 页面重组与批注导出算法

输入： 页面配置数组 *pages*，输出文件名 *name*

输出： 导出 PDF 对象地址

- 1: 清洗输出文件名并创建任务编号
 - 2: **for** 每个页面配置 *page* **do**
 - 3: 读取源 PDF 页面并应用旋转与顺序
 - 4: 按式 4.2 换算批注坐标
 - 5: 将文本、画笔、矩形和箭头写入页面
 - 6: **end for**
 - 7: 合并页面并写入 MinIO
 - 8: 返回 API 代理下载地址
-

4.5 数据结构设计

4.5.1 MinIO 对象结构

系统将 MinIO 作为文件对象的统一存储位置。用户上传的原始文件以对象名保存，文件夹通过路径前缀和 `.keep` 空对象表达；预览缓存保存到 `previews/` 前缀；转换结果与 PDF 导出结果保存到 `conversions/` 前缀。对象名不仅是存储键，也是前端目录和文件定位的主要依据，因此上传、移动、重命名和删除接口都必须维护对象名前缀的一致性。

对象存储占用沿用第 3 章式 3.1 的需求口径。总体设计阶段需要进一步明确对象集合的过滤规则：用户容量统计只聚合原始文件对象和用户可见产物，不把目录占位对象、预览缓存和转换中间文件混入容量展示。这样能够避免文件夹数量或预览次数影响用户对真实资料规模的判断。

4.5.2 Redis 状态结构

Redis 承担两类状态：一类是任务状态，包含任务标识、任务类型、当前状态、结果地址、错误信息和时间戳；另一类是事件日志和健康快照，用于管理端观察近期操作和服务状态。任务状态至少覆盖 **pending**、**processing**、**success** 和 **failed**，事件日志按时间线保存上传、预览、转换、PDF 导出和系统健康相关事件。第 3 章图 3.2 已概括 MinIO 与 Redis 的概念数据模型，本节进一步说明状态存储设计。

任务总量沿用第 3 章式 3.2 的需求口径。Redis 设计需要保证等待、处理中、成功和失败任务能够被独立统计，并且任务总量只来自当前业务任务集合，避免与 Spark 历史样本规模混淆。

任务状态从创建到终态存在明确的迁移约束。转换任务与 PDF 页面处理任务提交后先进入等待状态，后台任务启动后进入处理中状态，产物写入 MinIO 后进入成功状态，异常捕获后进入失败状态。状态变化同时触发事件日志写入，使任务监控、运行拓扑和近期事件列表能够读取同一条业务链路。图 4.7 描述了状态迁移与事件传播关系。

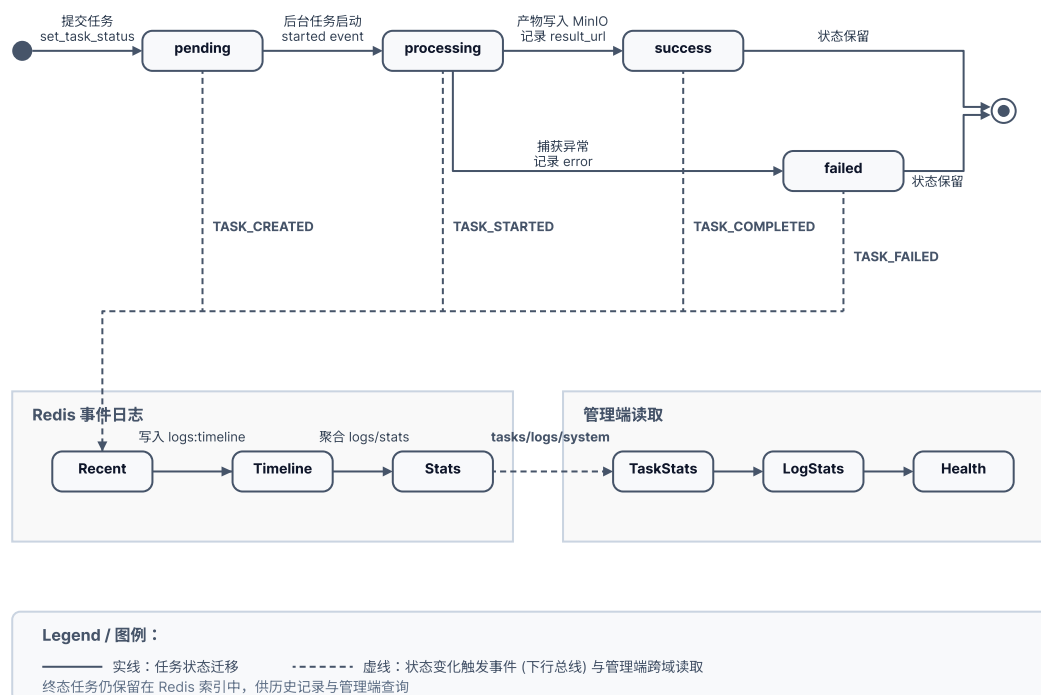


图 4.7 任务状态与事件流转图

4.6 管理驾驶舱数据管线设计

管理驾驶舱采用实时运行数据与历史分析样本并行的设计。实时运行数据直接来自 FastAPI 接口，适合反映当前文件数、任务数、失败数、队列长度、近期事

件和服务健康；历史分析样本来自 Spark 输出，适合表达较长窗口下的文件类型分布、转换矩阵、趋势变化和质量分析。图 4.8 展示了这一管线。该设计使用户端操作能够进入实时观察，同时保留大数据课程场景下的离线分析能力。

实时运行指标的计算以 Redis 中的任务集合与日志统计为基础。在式 3.2 的基础上，管理端第一屏的任务成功率与失败占比定义为：

$$R_s = \frac{C}{\max(T, 1)}, \quad R_f = \frac{F}{\max(T, 1)}. \quad (4.3)$$

其中 R_s 表示任务成功率， R_f 表示失败占比，分母使用 $\max(T, 1)$ 是为了避免任务总量为零时产生无意义除法。近 24 小时事件规模直接来自日志统计，记为 $L_{24} = E_{24}$ 。式 4.3 对应前端读取的 `/api/v1/tasks/stats`、`/api/v1/tasks/queue-length` 与 `/api/v1/logs/stats`，用于解释管理端实时数值会随业务操作发生变化。

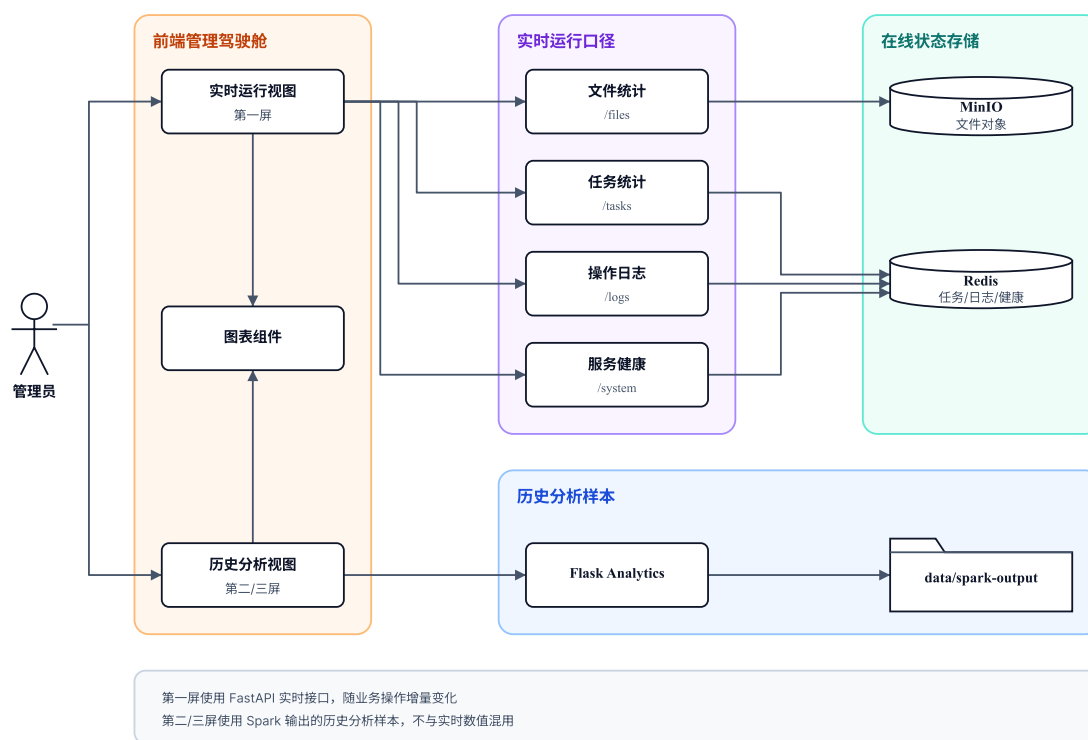


图 4.8 管理驾驶舱数据管线图

4.7 部署结构设计

系统采用本地进程管理与容器化基础依赖结合的部署方式。前端开发服务与 Flask 分析服务适合使用 PM2 管理，便于快速重启、查看日志和保留长期运行状态；Redis、MinIO、Gotenberg 等基础依赖适合由 Docker Compose 管理，确保端口、网络和数据卷稳定。FastAPI 主服务启动时初始化 MinIO、Redis 和健康检查循环，保证 API 可用性与基础依赖状态能够被系统状态接口观察到。

从启动顺序看，Redis、MinIO 和 Gotenberg 属于基础依赖，应先于 FastAPI 主服务启动；FastAPI 启动后负责创建存储桶、注册路由和启动健康检查循环；前端与 Flask Analytics 则可以作为独立展示服务由 PM2 持续守护。这样的部署边界使文件处理主链路与历史分析展示链路可以单独重启，降低调试管理端图表时影响用户上传和转换的概率。

从运行观测看，系统状态接口需要同时覆盖 API、对象存储、任务状态、转换服务和分析服务。若 MinIO 不可用，文件上传和下载会失败；若 Redis 不可用，任务状态和近期事件会失去持久记录；若 Gotenberg 不可用，Office 转 PDF 和部分预览链路会降级。将这些依赖纳入系统状态设计，可以让管理端显示的不只是页面是否打开，而是关键业务能力是否可用。

4.8 本章小结

本章完成了 CulCloud 的系统总体设计，明确了分层架构、模块边界、业务流程、数据结构、运行指标和部署结构。通过业务闭环、任务状态、数据模型和管理管线的组合设计，系统能够把用户端动作转换为可查询任务、可下载产物和可观察指标。

从总体架构看，系统采用前端界面、FastAPI 业务接口、领域服务、对象存储、运行状态和历史分析服务分层组织。该结构的好处是职责边界清晰：前端负责交互和状态展示，FastAPI 负责接口校验和业务编排，MinIO 负责文件对象保存，Redis 负责任务与日志状态，Gotenberg 负责重型文档转换，Spark 与 Flask Analytics 负责历史分析结果读取。不同模块之间通过 API 和对象键建立联系，而不是相互直接读取内部实现。

从业务流程看，本章把上传预览、格式转换、PDF 页面处理和管理端观察放入同一条闭环中分析。文件上传会形成对象和日志，文件预览会形成可复用缓存，转换和 PDF 导出会形成异步任务与结果产物，管理端再读取这些状态形成运行指标。这样的设计使系统不只是多个页面的集合，而是一个可追踪、可解释、可复现的资料处理平台。

从数据和运行角度看，系统没有引入复杂关系数据库，而是利用对象名前缀、任务状态集合和事件日志完成当前阶段所需的数据表达。该选择降低了本地部署和验证成本，但也要求在对象移动、目录删除、预览缓存和转换产物之间保持清晰边界。后续若引入用户权限、共享协作和长期审计，可以在现有对象键和任务标识基础上补充关系索引。

从设计边界看，本章确认的是最小可用平台的总体结构，而不是完整企业级

云盘方案。当前设计重点覆盖单用户资料管理、轻量文件处理、异步任务可追踪和管理端观察；对于细粒度权限、多人协同编辑、跨设备同步冲突、对象版本回滚和长期归档策略，只保留可扩展位置。这样能够让系统在课程实践规模内保持可运行、可说明，同时为后续增强留下清晰方向。

因此，第 4 章承担的是“把系统如何组织起来”讲清楚的任务，而不是替代后续实现说明。

下一章将在本章设计基础上进一步落到代码实现，说明各前端视图、API 客户端、后端路由、领域服务和数据对象之间如何协作；第 6 章则从功能用例、命令行结果和业务闭环三个层面验证这些设计是否真正落地。

第 5 章 系统详细设计及实现

本章从代码实现角度说明 CulCloud 的核心模块。系统前端主要位于 `file-cloud-frontend/src/views/`，统一 API 调用位于 `file-cloud-frontend/src/services/api.ts`；后端主入口位于 `backend/app/main.py`，业务路由位于 `backend/app/routers/`，领域服务位于 `backend/app/services/`。图 5.1 给出了详细设计组件协作关系，为后续模块说明提供总览。

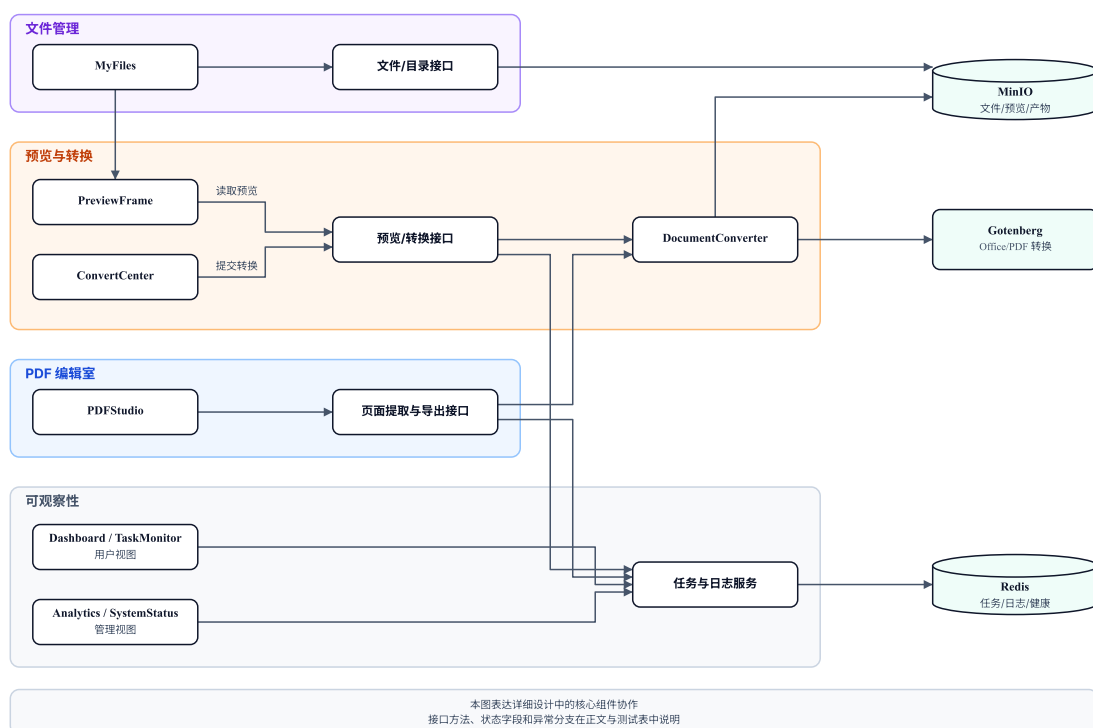


图 5.1 核心功能详细设计组件协作图

5.1 个人首页模块的实现

5.1.1 真实指标聚合

个人首页由 `Dashboard.tsx` 实现，主要展示文件数量、存储空间、近期任务和失败提醒。该模块不使用静态计数，而是通过 `services/api.ts` 中的 `listFiles`、`listTasks`、`getTaskStats` 等函数读取后端数据。文件数量和容量来自 `/api/v1/files` 返回的 MinIO 对象元数据，任务成功和失败数量来自 Redis 任务索引。当前设计的关键点是让首页指标跟随真实文件处理行为变化，避免用户端完

成转换后首页仍显示旧状态。

实现上，首页指标不直接读取浏览器缓存，而是把接口返回作为唯一可信来源。当前文件总量由递归文件列表聚合得到，任务数据由 `/api/v1/tasks/stats` 提供，近期失败可进一步通过 `/api/v1/tasks/recent-failures` 获取。这样处理的好处是页面刷新、前端重启或切换端侧后，指标仍能从后端状态恢复；缺点是首页依赖后端服务可用性，因此前端需要为接口异常提供明确提示，而不能默默显示零值。

5.1.2 失败原因回溯

失败任务提示依赖 `task_queue.py` 和 `task_tracker.py` 中的状态写入逻辑。当转换或 PDF 导出失败时，后端将错误原因写入任务对象的 `error` 字段，并通过任务接口返回给前端。前端在任务提示或历史列表中读取该字段，使用户能够区分格式不支持、文件损坏、转换服务不可用等不同故障来源。该实现使“失败计数”和“失败原因”具有同一后端来源。

从现场说明角度看，首页模块可以概括为“指标聚合层”：它本身不处理文件，也不执行转换，而是把文件管理、转换任务和异常状态聚合到一个用户可读入口。该设计降低了用户在多模块之间寻找结果的成本，也让后续管理端能够复用同一批任务与日志数据。

5.2 我的文件模块的实现

5.2.1 上传、目录与类型分类

我的文件模块由 `MyFiles.tsx` 实现，后端对应 `files.py` 与 `folders.py`。上传时，前端调用 `uploadFile(file,{prefix,relativePath})`，后端执行路径清洗、对象命名和 `MinIO` 写入，并通过 `log_event` 记录文件上传事件。目录能力并不依赖传统关系数据库表，而是使用对象名前缀和 `.keep` 空对象表达。前端通过 `currentPrefix` 维护当前目录，文件类型由扩展名映射到 PDF、文档、表格、PPT、图片、文本、压缩包和其他类型。

该模块的关键实现边界在于：前端只负责组织交互状态和展示分类，不直接操作对象存储；对象命名、路径合法性和文件内容写入均在后端完成。后端 `_sanitize_path` 会拒绝包含危险路径片段的输入，避免用户通过相对路径影响系统目录。文件上传完成后，前端重新拉取列表而不是本地拼接文件项，保证显示内容与 `MinIO` 中的实际对象保持一致。

上传后的容量展示采用对象大小聚合结果，与第 3 章式 3.1 保持一致。前端只

展示接口返回的文件大小和汇总容量，不把本地选择文件时的临时大小作为最终值。这样即使后端对文件名、路径或对象前缀进行了清洗，页面上看到的仍然是对象存储确认后的结果。

5.2.2 目录操作与对象移动

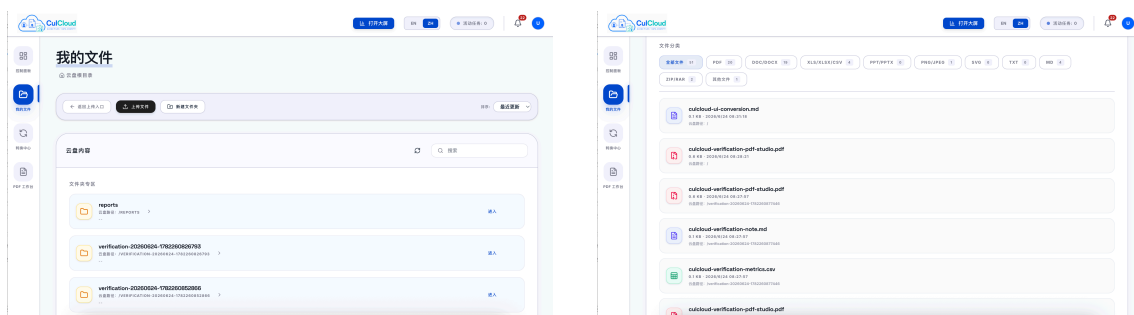
文件移动由 `moveToFolder` 调用 `/api/v1/folders/move` 完成。后端路由 `folders.py` 校验目标目录是否存在，并通过 `MinIO` 的复制和删除完成对象迁移。对文件夹删除、重命名和移动等操作，系统需要处理前缀递归关系，避免将父目录移动到自身子目录中造成路径循环。表 5.1 汇总了用户端主要模块与后端实现位置。

由于对象存储没有真正的目录表，所有目录操作最终都会落到对象名前缀变化上。系统把“路径前缀”作为目录语义，把“对象名”作为文件唯一定位依据，使文件夹浏览、文件移动和文件下载可以共享同一套对象模型。这种设计简化了数据层，但要求后端在移动和删除时格外注意级联对象范围，避免误删预览缓存或转换产物。

表 5.1 核心模块代码位置与数据链路

模块	前端位置	后端位置	主要数据对象
我的文件	<code>MyFiles</code>	文件/目录路由	<code>MinIO</code> 文件对象、目录前缀
文件预览	<code>PreviewFrame</code>	预览路由与预览服务	预览类型、预览缓存
转换中心	<code>ConvertCenter</code>	转换路由与转换服务	<code>Redis</code> 任务、转换产物
PDF 编辑室	<code>PDFStudio</code>	PDF 处理接口与转换服务	页面配置、批注对象、PDF 产物
管理驾驶舱	<code>Analytics</code>	任务、日志、健康与分析接口	实时指标、历史分析样本

图 5.2 展示了文件管理模块的运行结果。左图为根目录与文件夹视图，右图为上传后的文件列表和类型筛选视图。可以看到，文件对象进入列表后保留名称、大小、更新时间和格式标识，说明前端列表与 `MinIO` 对象元数据已经建立映射。



(a) 目录与文件夹视图

(b) 上传后文件列表

图 5.2 文件管理模块界面结果图

5.3 文件预览模块的实现

5.3.1 预览类型识别

文件预览由前端 `PreviewFrame` 和后端 `preview.py`、`services/preview.py` 协同实现。后端 `get_preview_metadata` 根据扩展名判断 `preview_type`，并返回 `content_url`。PDF、图片、音频和视频可直接通过内容接口返回；TXT 使用文本响应；Markdown 和 CSV 转换为隔离 HTML；Office 文件通过 `Gotenberg` 或转换服务生成 PDF 缓存。该设计使预览模块不要求前端理解所有文件解析细节，前端只需根据 `preview_type` 选择对应展示组件。

预览接口分为元数据接口与内容接口两层。元数据接口负责说明文件是否可预览、预览类型和内容地址；内容接口负责返回字节流或渲染后的 HTML。这样做可以让前端先决定展示容器，再按需加载实际内容，避免大文件在列表阶段被提前读取。对于 Office 文件，缓存对象名由源文件信息计算得到，重复预览时优先复用缓存，减少转换服务压力。

第 4 章算法 4.1 已给出预览分派规则。本章关注其实现位置：扩展名识别、内容地址生成和不可预览状态由后端完成，前端只根据返回的 `preview_type` 选择 PDF、图片、音视频、文本或 HTML 容器。

预览分派还承担异常兜底职责。若文件类型无法识别，接口应明确返回不可预览状态，而不是让前端反复尝试打开内容地址；若 Office 预览缓存生成失败，页面应保留下载入口和错误提示，避免用户误以为文件丢失；若 Markdown 或 CSV 内容较长，前端只负责展示后端生成的隔离内容，不把整份文本写入全局状态。这样可以把解析风险限制在预览模块内部。

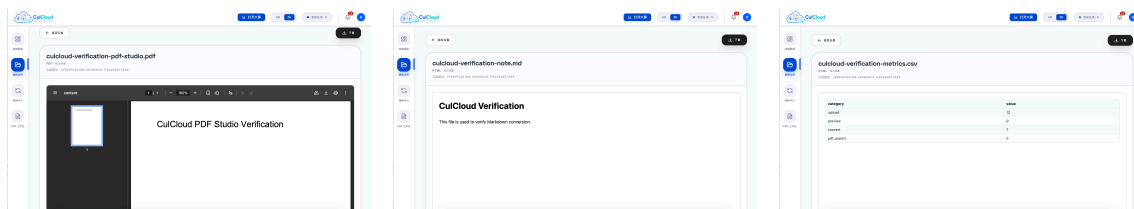
缓存策略也影响用户体验。对于同一对象的重复预览，系统优先复用已生成的 PDF 或 HTML 缓存；当源文件发生移动或重命名时，内容接口仍应以对象名为

准重新定位文件。该设计让预览模块既能减少转换服务压力，又能保持与文件管理模块的一致性。

5.3.2 结构化内容渲染

Markdown 和 CSV 的预览是该模块的关键实现。Markdown 需要在后端转换为 HTML，同时避免用户上传内容直接污染主页面；CSV 需要读取字节流、处理编码并生成表格化视图。图 5.3 展示了 PDF、Markdown 和 CSV 三类预览结果，三者分别对应二进制文档、标记文本和结构化表格。

从实现风险看，CSV 与 Markdown 预览比图片和 PDF 更容易出现编码、HTML 安全和长内容排版问题。因此后端采用独立响应内容承载渲染结果，前端通过预览框加载，既保留可读性，也减少对主页面状态树的影响。该策略适合课程资料中常见的配置文件、实验数据表和说明文档。



(a) PDF 预览

(b) Markdown 预览

(c) CSV 预览

图 5.3 文件预览模块界面结果图

5.4 转换中心模块的实现

5.4.1 白名单与能力矩阵

转换中心由 `ConvertCenter.tsx` 实现，后端对应 `routers/convert.py` 和 `services/converter.py`。系统在前后端都维护 P0 白名单：前端 `P0_WHITELIST` 控制用户可选择的转换项，后端 `_assert_p0_whitelist` 在 API 层再次拒绝未开放格式。这样即使前端被绕过，后端也不会执行未验证的转换路径。转换能力信息由 `/api/v1/convert/capabilities` 返回，便于前端按文件扩展名过滤目标格式。

白名单设计体现了系统的能力边界。项目并没有把所有库中理论可调用的转换函数都暴露给用户，而是只开放经过页面、接口和产物验证的高频链路。前端负责降低误操作概率，后端负责最终安全约束。该双层约束可以在后续扩展格式时逐项放开，而不会影响已经稳定的转换路径。

5.4.2 异步任务执行

转换任务提交后，后端调用 `set_task_status(task_id, pending)` 写入初始状态，并通过 FastAPI BackgroundTasks 调用 `async_convert_task`。后台任务读取源文件，执行 DocumentConverter 中对应方法，例如 `word_to_pdf`、`csv_to_pdf`、`markdown_to_pdf`、`svg_to_png` 等。成功后，结果写入 MinIO 的 `conversions/` 前缀，任务状态更新为 `success` 并返回 API 代理下载地址；失败时写入 `failed` 与错误原因。图 5.4 展示了转换完成后的当前任务状态。

第 4 章算法 4.2 概括了转换与 PDF 导出共用的异步处理模式。代码实现中，转换中心对应 `async_convert_task`，PDF 编辑室对应 `async_pdf_process_task`，二者都通过任务状态写入函数保持等待、处理中、成功和失败的一致语义。这样前端轮询、任务监控和管理驾驶舱读取到的是同一套状态来源。

前端轮询并不是无限循环，而是以第 4 章式 4.1 作为终止条件。`ConvertCenter.tsx` 和 `PDFStudio.tsx` 在读取到成功或失败终态后停止定时查询，避免任务已经结束后仍持续请求后端。

转换结果下载采用 API 代理地址，而不是直接暴露 MinIO 内部地址。这样可以避免浏览器跳转到对象存储端口后出现 Office 压缩包内部 XML 被直接打开等异常体验，也让下载行为继续受后端接口控制。任务状态轮询由前端定时触发，轮询终止条件是任务进入成功或失败终态，避免长期无意义请求。

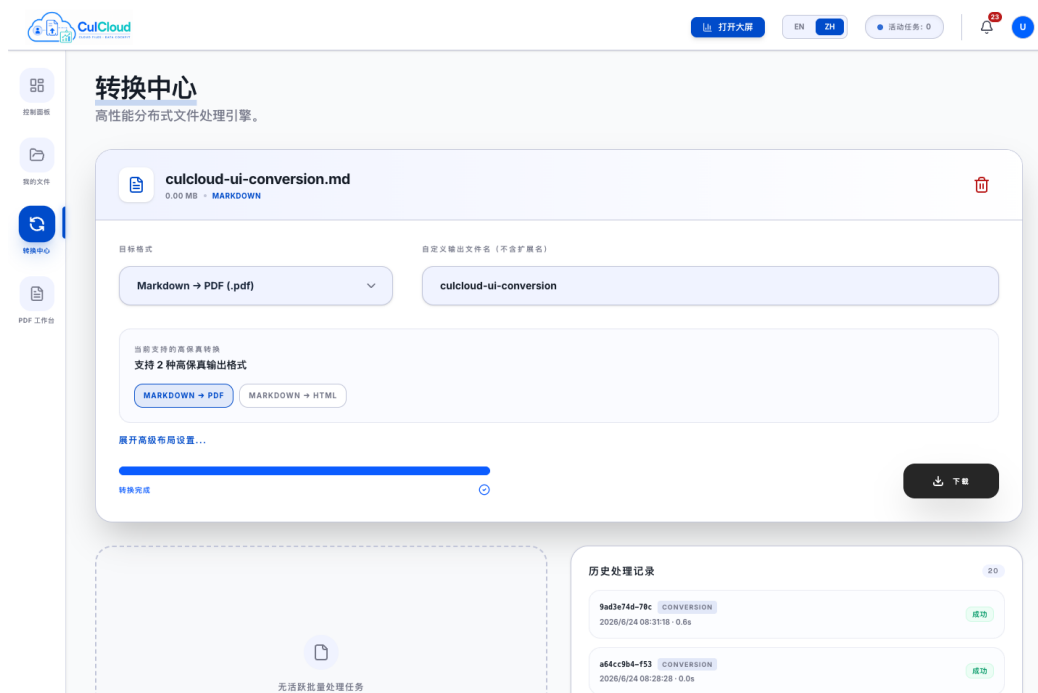


图 5.4 转换中心任务完成界面结果图

5.5 PDF 编辑室模块的实现

5.5.1 页面工作区

PDF 编辑室由 `PDFStudio.tsx` 实现，核心状态包括 `WorkspacePage`、`Annotation`、当前工具、缩放比例和选中对象。用户选择 PDF 后，前端调用 `extractPdfPages`，后端使用 `DocumentConverter.pdf_to_images` 将每页转换为 PNG，并保存到 `previews/{preview_id}/page_N.png`。前端据此构建页面工作区，支持页面选择、排序、旋转、删除和批注。

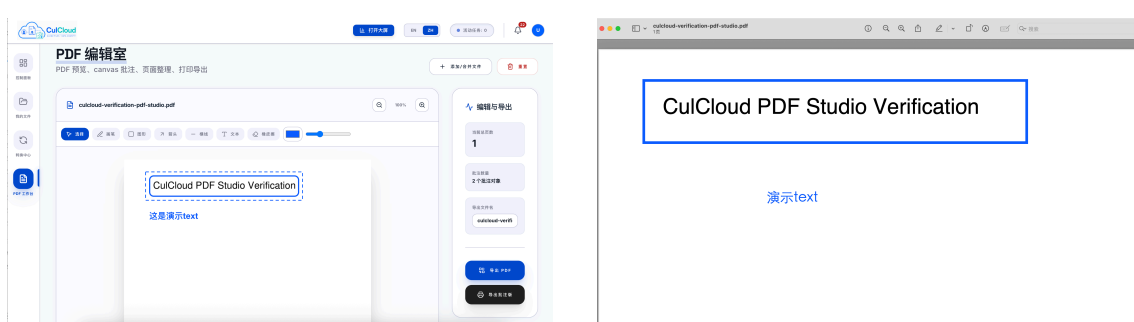
PDF 工作区的设计采用“页面数组 + 批注数组”的状态模型。页面数组保存源对象、页码、旋转角度和尺寸信息，批注数组保存工具类型、坐标点、文本和颜色。排序与删除操作只改变页面数组，不直接修改源 PDF；导出时再由后端根据最终配置生成新文件。该方式使编辑过程可撤回、可重组，也避免频繁写入大体积 PDF 文件。

5.5.2 批注与导出

批注对象记录工具类型、坐标点、文本、颜色和线宽。导出时，前端将页面配置和批注数组提交给 `processPdf`，后端 `async_pdf_process_task` 调用 `DocumentConverter.process_pdf_pages` 重组 PDF 并写入批注。图 5.5 展示了 PDF 编辑工作区和导出后文件结果。该组图说明页面操作不是停留在浏览器状态，而是能够生成可打开的新 PDF 文件。

批注导出的难点在于坐标映射。前端操作发生在缩放后的画布上，后端写入发生在 PDF 页面坐标系中，因此导出配置必须携带前端画布宽高，并在后端按比例换算。对于文本批注，还需要加载适合中英文混排的字体，保证导出文件中的中文文本不会出现字符间距异常。

第 4 章式 4.2 和算法 4.3 已给出坐标映射与导出规则。实现层面，前端提交页面配置数组和批注数组，后端在 `process_pdf_pages` 中读取源页面、应用旋转和顺序、换算批注坐标、加载 CJK 字体并写入新 PDF。这样 PDF 编辑室的交互状态能够转化为可打开的文件产物。



(a) 页面与批注工作区

(b) 导出后文件结果

图 5.5 PDF 编辑室处理结果图

5.6 管理驾驶舱模块的实现

管理驾驶舱由前端 `Analytics.tsx`、`AnalyticsSlides.tsx` 以及后台接口服务共同构建，旨在实现“实时系统运行状态监控”与“历史大数据遥测分析”的双流治理结构。

5.6.1 实时运行视图设计与实现

实时运行视图（第一屏）主要向管理人员展示当前系统实例的活跃状态、事件流与微服务拓扑，该页面通过前端组件定时轮询读取后端接口提供数据支撑。在图表设计上，微服务拓扑图作为核心监控组件，采用有向拓扑图描述服务链路，在前端配置固定二维坐标描述各个节点的位置（如网关节点位于 `[368, 190]`，Gotenberg 转换服务位于 `[202, 282]`），并基于服务权值动态绑定节点尺寸，结合图形底图绘制四层渐变圆环以达到深色背景下的发光脉冲效果。对各组件探测延迟使用 `latency_ms` 进行度量。该图表的数据直接来源于后端的健康探测接口 `/api/v1/system/health`，该接口由后端定时对 Redis、MinIO 以及 Gotenberg 等下游组件进行 TCP 连通性测试并返回延迟快照，从而能够在系统出现部分断点或通信降级时实时呈现为红色警示，为系统管理员提供直观的分布式连接诊断手段。而对于事件流的监测，系统配置了二十四小时事件趋势折线面积图，为了保持界面在空间紧凑下的极简展示，该图表移除了坐标轴，通过 `grid: {top: 8, bottom: 8, left: 4, right: 4}` 消除边界留白，利用带有垂直渐变色填充的平滑曲线展示流量的波峰与波谷。其数据直接从 Redis 事件数据库接口 `/api/v1/logs/timeline` 中的时间戳按小时聚合而来，能为后续的容器副本弹伸与资源调整提供定量趋势依据。此外，实时操作分布环形图采用环形饼图展示会话中用户各项业务操作的占比，配置 `radius: ['35%', '60%']` 与 `center: ['50%', '55%']` 参数，其标签格式配置为 `'{b}\n{d}%'`，其数据来自于 Redis 实时记录的多维度操作统计接口

/api/v1/logs/stats，能帮助运维人员快速辨识当前系统整体的读写与算力吞吐属性。

图 5.6 展示了管理驾驶舱的实时运行监控界面。可以看到，系统拓扑网络、近期系统操作日志时间流、当前存活服务运行状况和实时操作分布饼图能够同步更新，完成了平台运行态势的直观呈现。

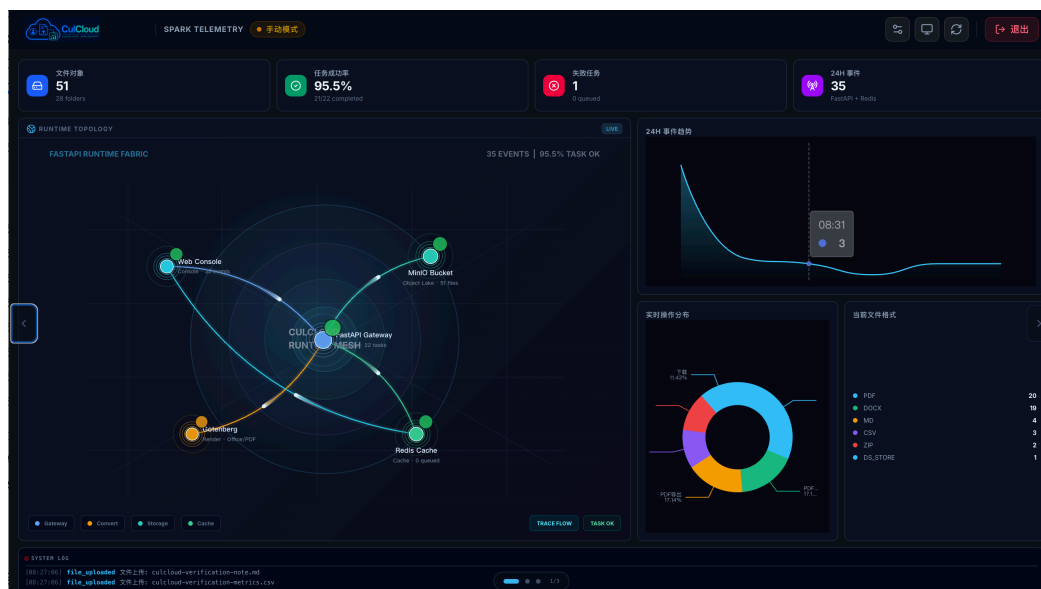


图 5.6 管理驾驶舱实时运行拓扑与操作事件监控界面

5.6.2 历史分析视图设计与实现

历史分析视图（第二、三屏）用于展现离线大数据分析管线的聚合计算成果，其数据接口由 Flask 分析服务提供，通过对 Spark 产生的 JSON 规范契约进行离线读取来实现。首先，历史文件类型分布图采用带有垂直渐变色彩的纵向柱状图来展示不同格式文件的绝对体量，其数据来自于对 MinIO 中所有存量文件对象元数据的离线聚合。这能直接指导系统对 Word、PDF 等高频格式进行预缓存优化，从而提升用户的预览打开速度。历史操作分布实心饼图则在侧边配合了等宽字体的操作次数图例清单，并在饼图中心区域通过绝对定位渲染总操作数文字，该图表基于 Spark 对历史遥测记录的分组累加结果，直接体现了平台的累计用户活跃度与功能的真实使用频次。为了清晰表现异构文件格式之间的流转态势，系统引入了转换桑基图，其配置采用三十二次迭代布线算法，并巧妙地通过为源格式与目标格式加上命名空间前缀（如 `source:docx` 和 `target:pdf`）来将双向循环转换模型解耦为单向无环的图结构，避免了有向环图导致图表绘制死锁的风险。其数据源自对历史格式转换日志中成功链条的 Spark 分析结果，这有助于管理团队向分析接口 /api/v1/analytics/conversion-stats 查询转换流向瓶颈，为对应的算力

调度和格式白名单扩容提供客观支持。图 5.7 展示了离线文件处理与转换分析可视化界面，展现了存量文件大小、耗时分布及桑基图等分析成果。

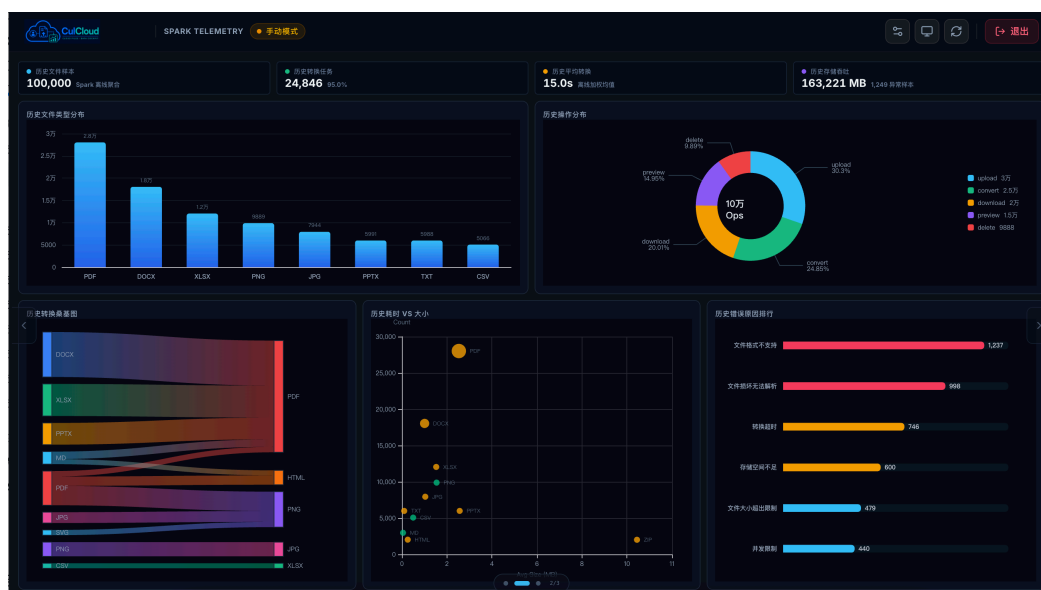


图 5.7 管理驾驶舱离线文件处理与转换分析可视化界面

而针对转换流程中的效率，历史耗时与文件大小散点图将 X 轴设为文件大小，Y 轴设为平均转换时长，气泡的物理尺寸 (symbolSize) 正比于并发样本规模，并配置三种色彩标识不同的成功率阈值。该数据直接源于离线遥测的耗时明细日志，有利于快速锁定导致引擎死循环或高负荷卡顿的非典型文件格式。同时，为了直观汇总历史处理中的异常成因，系统使用水平横向条形图展现具体的错误类别排行，并配置了半透明的填充背景，以警示性色彩从高到低呈现格式不支持、超时和溢出等故障的排序，其数据来自于对 Celery 任务失败原文字段的过滤分组，直接指导了白名单机制的策略收紧与转换服务引擎的补丁更新。另外，历史存储水位仪表盘采用单指针指示表，量程对应两百吉字节的总空间，并在表针扫过区域呈现渐变彩条，表盘中心直接显示当前占用百分比与容量数值，数据来自 Spark 统计的 MinIO 物理存储字节累加，能够及时提示管理团队在水位进入 85% 临界线时进行存储节点的水平扩展。

最后，为了监控分析指标本身的可信度，系统集成历史数据质量雷达图和数据质量仪表盘，前者基于完整性、一致性、时效性等五个轴向对遥测样本进行评估，后者则直观呈现了有效遥测记录占原始总条数的百分比，其数据源自 Spark 离线清洗管线的输出比对指标，用以量化整个日志采集链路的数据损耗，确保驾驶舱中展示的一切可视化图表结论不受脏数据污染。而为了进一步掌握错误发生的时空规律，系统还集成了七乘二十四小时错误热力图，横轴对应全天的二十四

小时段，纵轴为星期，用由深蓝向明红的色彩过渡展现错误计数的密集度，这非常有助于管理员排查如每日凌晨定时备份或其他计算批处理造成的瞬时高错误热点。图 5.8 展示了服务监控与遥测数据质量管理分析界面，呈现了错误热力图、服务状态雷达图和多维质量考核结果。

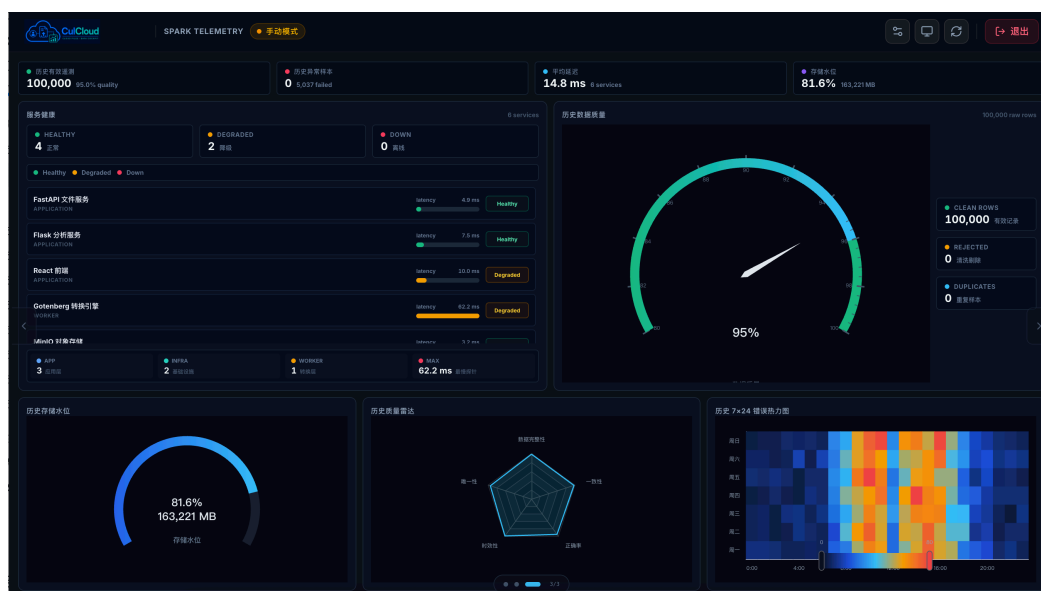


图 5.8 管理驾驶舱服务监控与遥测数据质量管理分析界面

5.7 本章小结

本章围绕真实代码位置说明了个人首页、我的文件、文件预览、转换中心、PDF 编辑室和管理驾驶舱的实现方式。系统实现的核心特点是前端视图、API 客户端、后端路由、领域服务、MinIO 对象和 Redis 状态之间形成清晰链路，使用户端操作能够产生可查询任务、可下载产物和管理端可观察数据。

第 6 章 系统测试

本章围绕 CulCloud 的核心业务链路进行测试，重点验证文件上传、目录整理、文件预览、格式转换、PDF 页面导出、任务状态、日志记录和管理端观察是否能够形成可复现闭环。测试结果既包括浏览器端界面结果，也包括后端自动化测试、前端类型检查、运行接口探测和端到端角色路由验证。

6.1 测试环境

6.1.1 软硬件环境

测试环境采用本地开发机、FastAPI 服务、MinIO、Redis、Gotenberg、React 前端和 Flask/Spark 分析服务组成。表 6.1 给出了主要配置。该环境能够覆盖用户端、管理端、对象存储、任务状态、日志和历史分析服务之间的真实交互。

表 6.1 测试环境配置

类别	技术组成	测试关注点
前端	React、TypeScript、Vite	页面渲染、路由隔离、API 调用、状态刷新
后端	FastAPI、Pydantic	路由参数、白名单校验、任务状态返回
存储	MinIO	文件对象、目录前缀、预览缓存、转换产物
状态	Redis	任务状态、任务索引、事件日志、健康快照
转换	Gotenberg、PyMuPDF、本地库	Office/PDF/图片/Markdown/CSV 转换与 PDF 页面处理
分析	Spark、Flask、ECharts	历史分析样本读取与图表展示

6.1.2 测试数据准备

测试数据覆盖 PDF、Markdown、CSV、图片、Office 文档和 PDF 编辑样例。PDF 用于验证浏览器预览、页面提取和导出；Markdown 用于验证文本转 PDF；CSV 用于验证表格化预览；图片和 Office 文件用于验证预览类型识别与转换边界。测试还准备了包含中文文本和批注框的 PDF 文件，用于检验导出后文字、边框和页面位置是否保持一致。

6.2 测试层次与边界

系统测试按环境层、接口层、业务层、联动层和异常层展开。环境层验证 Docker Compose 与 PM2 管理的服务是否在线；接口层验证 FastAPI、Flask Analytics 和健康检查是否返回结构化结果；业务层验证上传、预览、转换和 PDF 导出是否形成产物；联动层验证用户端操作是否进入任务统计、日志统计和管理端实时视图；异常层验证不存在的资源、非白名单格式和依赖异常能否被解释。表 6.2 给出了测试层次。

表 6.2 系统测试层次划分

层次	验证对象	关键边界	判定标准
环境层	Docker Compose、PM2	容器依赖与本地进程边界	服务列表中关键组件处于运行状态
接口层	FastAPI、Flask Analytics	HTTP API 与分析服务边界	健康、统计、列表接口返回 JSON
业务层	文件与 PDF 处理	用户操作与后端任务边界	任务进入终态并生成可访问产物
联动层	用户端与管理端	业务增量与实时图表边界	任务、日志和指标随操作刷新
异常层	非法资源与格式	错误输入与结构化响应边界	返回错误原因而非页面崩溃

从系统边界看，单纯打开页面不能证明服务链路完整，因此测试需要同时保留命令行结果和浏览器端结果。命令行结果用于证明服务进程、容器依赖、接口返回和自动化用例可重复；浏览器端结果用于证明用户能够完成实际操作。两类证据共同覆盖“系统是否运行”和“业务是否闭环”两个问题。

环境层测试的重点是确认基础依赖和应用进程处于同一可用窗口内。Docker Compose 负责 Redis、MinIO、Gotenberg 等基础服务，如果这些容器未启动，即使前端页面能够打开，也无法完成任务状态写入、对象保存和文档转换。PM2 负责守护前端、后端或分析服务，若进程状态异常，接口调用和图表刷新也会失效。因此环境层测试先于功能测试执行，只有依赖和进程正常，后续接口层和业务层结果才具有解释意义。

接口层测试的重点是确认系统之间的边界是否真实连通。健康接口用于判断服务是否可访问，文件接口用于判断对象存储链路是否可读，任务统计接口用于判断 Redis 任务集合是否可读，日志统计接口用于判断事件留痕是否可读，分析接

口用于判断历史样本是否能被 **Flask** 服务读取。若这些接口返回结构化 **JSON**，就说明系统内部边界已经具备可观测性；若只依赖页面结果，则很难定位故障发生在前端、后端还是基础依赖。

联动层测试是本章最重要的验证点。用户端完成上传、转换或 **PDF** 导出后，系统应产生对象、任务和日志三类变化；管理端第一屏应通过实时接口读取这些变化，而不是通过静态样本显示固定数值。该测试能够证明用户端业务数据与管理端运行观察之间存在真实通路，也能证明历史分析页与实时运行页的口径已经分离。

6.3 功能测试

6.3.1 云盘与预览测试

云盘测试重点验证文件对象是否能正确进入 **MinIO**，并在用户端列表中以目录和类型分类形式呈现。预览测试重点验证后端能否根据扩展名返回正确预览类型，前端能否使用对应渲染组件展示内容。表 6.3 给出了主要用例。

表 6.3 云盘与预览功能测试用例

编号	模块	输入	操作	预期结果
T01	我的文件	PDF/MD/CSV 文件	上传至根目录	文件对象写入 MinIO ，列表显示名称、大小和类型
T02	我的文件	文件夹名称	新建文件夹	生成 .keep 目录标记并显示文件夹项
T03	我的文件	已上传文件	移动到目标目录	对象名前缀更新，刷新后路径正确
T04	文件预览	PDF 文件	打开预览页	浏览器内嵌 PDF 内容正常显示
T05	文件预览	Markdown 文件	打开预览页	后端返回 HTML ，前端展示标题与正文
T06	文件预览	CSV 文件	打开预览页	表格字段和数据行按表格展示

测试结果表明，文件上传后能够进入对象列表，预览页能够按类型渲染 **PDF**、**Markdown** 和 **CSV**。对于 **Office** 等需转换的文件，系统采用预览缓存方式避免重复

转换；对于暂不支持预览的格式，接口返回明确状态，前端展示可理解提示。

6.3.2 转换链路测试

转换链路测试以当前 P0 白名单为准，覆盖 Word 到 PDF、DOC 到 PDF、Excel 到 PDF、CSV 到 PDF、PPTX 到 PDF、Markdown 到 PDF、Markdown 到 HTML、SVG 到图片/PDF、图片到 PDF、图片到 ICO、PDF 到图片等链路。测试重点不只是接口能否返回任务标识，还要验证任务是否经历排队、处理中、成功或失败状态，并最终生成可下载产物。表 6.4 给出了转换链路测试项。

表 6.4 格式转换链路测试用例

编号	输入类型	目标格式	关键接口	验证点
C01	Markdown	PDF	/convert/existing	任务完成后生成 PDF 下载地址
C02	CSV	PDF	/convert/existing	表格内容进入转换产物
C03	DOC/DOCX	PDF	/convert	Gotenberg 转换服务可用
C04	PPTX	PDF	/convert	PPT 文件可转换为 PDF
C05	PNG/JPG	PDF	/convert	图片文件生成 PDF 页面
C06	非白名单格式	任意目标	/convert	后端拒绝并返回错误原因

6.3.3 PDF 编辑测试

PDF 编辑测试覆盖页面提取、缩略图切换、页面旋转、页面删除、页面排序、文本批注、矩形批注和导出。该模块的重点是验证前端工作区状态能否被后端准确解释，并输出新 PDF。表 6.5 给出了主要测试项。

表 6.5 PDF 编辑室测试用例

编号	操作	前端状态	后端处理	预期结果
P01	提取页面	页面数组	pdf_to_images	返回每页图片地址和页面尺寸
P02	页面排序	页码顺序变化	页面配置数组	导出文件页序与前端一致
P03	添加文本	文本批注对象	坐标映射写入 PDF	导出文件保留文本内容
P04	添加矩形	矩形批注对象	坐标映射写入 PDF	导出文件保留边框形状
P05	导出文件	输出文件名	写入 conversions/	返回下载入口并可打开查看

6.4 系统边界验证

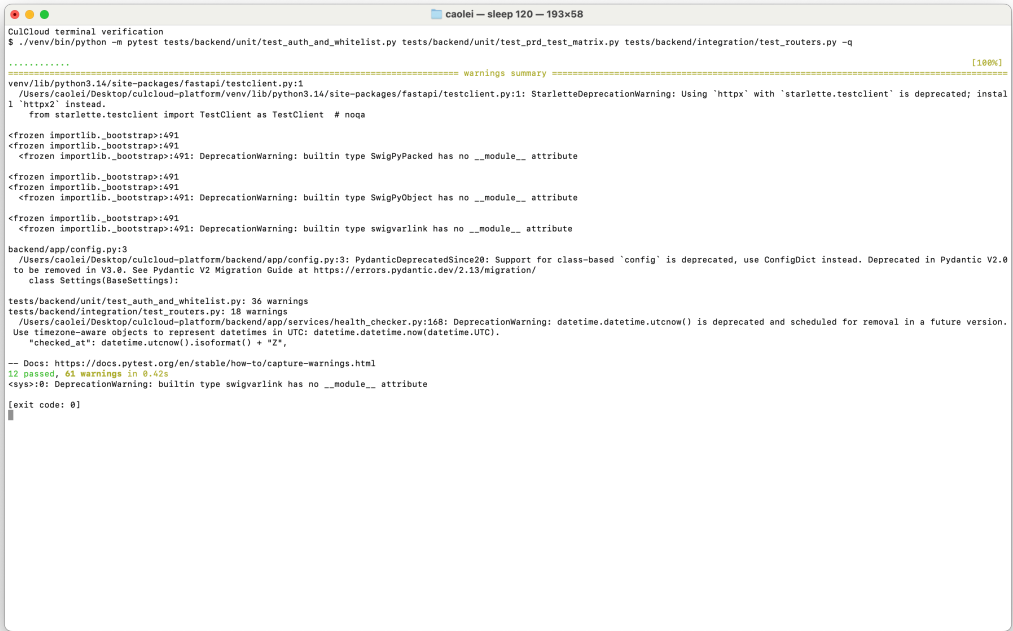
为保证测试结果可复现，系统补充了命令行验证。表 6.6 列出了命令、覆盖对象和结果含义。完整命令清单整理在 docs/02-process/scripts/manual_terminal_verification_checklist.sh 中，便于按同一顺序重复执行。

命令行验证与浏览器端结果互为补充。浏览器端可以证明用户是否能够完成操作，命令行结果则能证明后端用例、类型系统、接口返回和角色路由是否处于可重复验证状态。二者结合后，可以避免只看页面效果而忽略后端状态，也能避免只看接口返回而无法说明业务闭环。

表 6.6 系统边界命令与验证对象

编号	验证对象	典型命令	验证含义
R01	容器依赖	<code>docker compose ps</code>	Redis、MinIO、Gotenberg 等依赖处于运行状态
R02	本地进程	<code>pm2 status</code>	前端、后端和分析服务由进程管理器守护
R03	健康接口	<code>curl /health</code>	FastAPI 与分析服务可访问
R04	业务接口	<code>curl /files、/tasks/stats、/logs/stats</code>	文件、任务和日志数据可被实时读取
R05	自动化用例	<code>pytest、npm run lint、playwright</code>	后端、前端类型和端到端流程可重复验证
R06	异常边界	<code>curl 访问不存在资源</code>	系统返回结构化错误信息

图 6.1 展示了后端自动化测试运行结果。该组用例覆盖角色接口、白名单策略、PRD 功能矩阵和核心路由集成逻辑，结果为 12 个用例全部通过。图中存在第三方依赖告警，但未影响当前测试结论，后续升级依赖时仍需关注兼容性变化。



```
CuICloud terminal verification
$ ./venv/bin/python -m pytest tests/backend/unit/test_auth_and_whitelist.py tests/backend/unit/test_prd_test_matrix.py tests/backend/integration/test_routers.py -q

..... [100%]
===== warnings summary =====
venv/lib/python3.14/site-packages/fastapi/testclient.py:3
/Users/caolei/Desktop/cuicloud-platform/venv/lib/python3.14/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa

<frozen importlib._bootstrap>:491
<frozen importlib._bootstrap>:491
<frozen importlib._bootstrap>:491: DeprecationWarning: builtin type SwigPyPacked has no __module__ attribute

<frozen importlib._bootstrap>:491
<frozen importlib._bootstrap>:491
<frozen importlib._bootstrap>:491: DeprecationWarning: builtin type SwigPyObject has no __module__ attribute

<frozen importlib._bootstrap>:491
<frozen importlib._bootstrap>:491
<frozen importlib._bootstrap>:491: DeprecationWarning: builtin type swigvarlink has no __module__ attribute

backend/app/config.py:3
/Users/caolei/Desktop/cuicloud-platform/backend/app/config.py:3: PydanticDeprecatedSince20: Support for class-based `config` is deprecated, use ConfigDict instead. Deprecated in Pydantic V2.0
to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.13/migration/
  class Settings(BaseSettings):

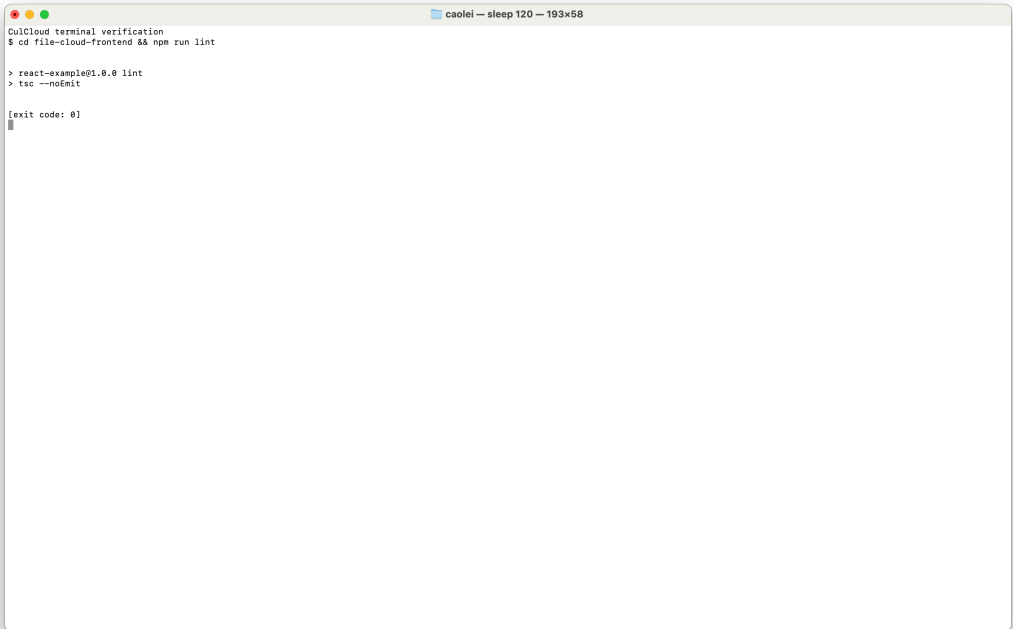
tests/backend/unit/test_auth_and_whitelist.py: 36 warnings
tests/backend/integration/test_routers.py: 18 warnings
/Users/caolei/Desktop/cuicloud-platform/backend/app/services/health_checker.py:168: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version.
Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  "checked_at": datetime.utcnow().isoformat() + "Z",

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
13 passed, 45 warnings in 8.42s
<sys>:0: DeprecationWarning: builtin type swigvarlink has no __module__ attribute

[exit code: 0]
```

图 6.1 后端自动化测试运行结果

图 6.2 展示了前端 TypeScript 静态检查结果。该结果说明当前前端视图、API 类型和状态管理代码不存在阻塞编译的类型错误。



```
CuICloud terminal verification
$ cd file-cloud-frontend && npm run lint

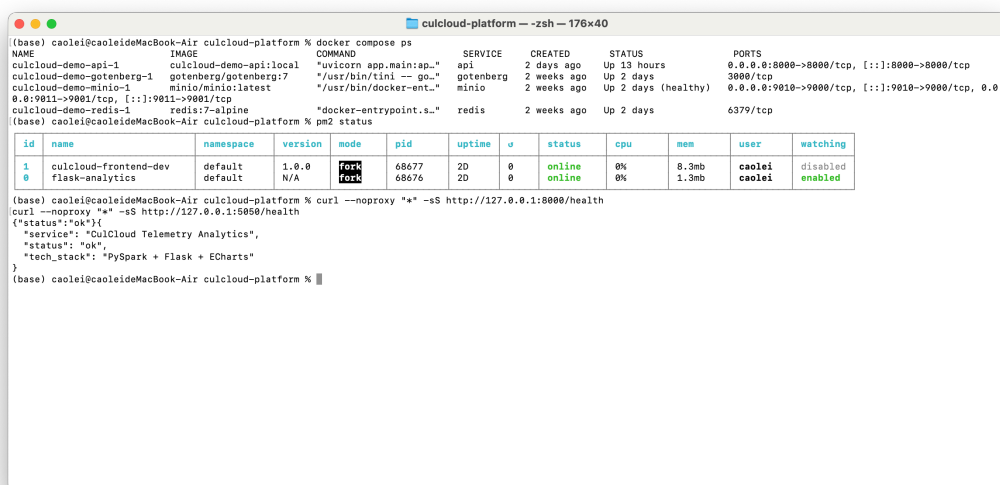
> react-example@1.0.0 lint
> tsc --noEmit

[exit code: 0]
```

图 6.2 前端 TypeScript 静态检查结果

图 6.3 展示了服务边界探测与接口健康返回结果。该结果用于确认 Docker

Compose 依赖容器、PM2 应用进程以及 FastAPI/Flask 主服务均处于正常运行与可访问状态。



```
(base) caolei@caoleideMacBook-Air culcloud-platform % docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED   STATUS    PORTS
culcloud-demo-api-1  culcloud-demo-api:local            "uvicorn app.main:app"              api        2 days ago    Up 13 hours    0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
culcloud-demo-gutenberg-1  gotenberg/gutenberg:7            "/usr/bin/tini -- go-"            gutenberg  2 weeks ago    Up 2 days    3000/tcp
culcloud-demo-minio-1  minio/minio:latest                "/usr/bin/docker-ent-"            minio      2 weeks ago    Up 2 days (healthy)    0.0.0.0:9010->9000/tcp, [::]:9010->9000/tcp, 0.0.0.0:9011->9001/tcp, [::]:9011->9001/tcp
culcloud-demo-redis-1  redis:7-alpine                    "docker-entrypoint.s-"            redis      2 weeks ago    Up 2 days    6379/tcp

(base) caolei@caoleideMacBook-Air culcloud-platform % pm2 status

```

id	name	namespace	version	mode	pid	uptime	v	status	cpu	mem	user	watching
1	culcloud-frontend-dev	default	1.0.0	fork	68677	2D	0	online	0%	8.3mb	caolei	disabled
0	flask-analytics	default	N/A	fork	68676	2D	0	online	0%	1.3mb	caolei	enabled

```
(base) caolei@caoleideMacBook-Air culcloud-platform % curl --noproxy "*" -sS http://127.0.0.1:8000/health
{"status":"ok"}
{"service": "CulCloud Telemetry Analytics",
"status": "ok",
"tech_stack": "PySpark + Flask + ECharts"
}
(base) caolei@caoleideMacBook-Air culcloud-platform %
```

图 6.3 接口与服务边界探测运行结果

图 6.4 展示了端到端自动化测试运行结果。该测试套件包含 11 个用例，全方位验证了角色路由隔离、白名单策略、命名记忆、Excel 预设参数简化以及 MinIO 上传删除的端到端业务闭环，结果为 11 个用例全部顺利通过。



```
(base) caolei@caoleideMacBook-Air culcloud-platform % cd /Users/caolei/Desktop/culcloud-platform
npm run test:e2e

> test:e2e
> PLAYWRIGHT_USE_SYSTEM_CHROME=true npm --prefix tests/e2e run test

> culcloud-e2e@1.0.0 test
> playwright test

Running 11 tests using 5 workers
11 passed (5.9s)

To open last HTML report run:
npx playwright show-report

(base) caolei@caoleideMacBook-Air culcloud-platform %
```

图 6.4 端到端自动化测试运行结果

图 6.5 展示了系统对于非法请求与不存在资源的异常响应边界测试。当请求不存在的任务 ID 和文件 ID 时，后端返回了结构化的 JSON 错误原因而没有引起服务崩溃，证明异常边界响应机制运行正常。



图 6.5 不存在资源与异常请求的错误响应边界结果

6.5 问题处理与修复验证

6.5.1 文件移动路径边界

文件移动测试中重点检查根目录、同名目录和子目录移动边界。系统通过 `folders.py` 对目标前缀进行规范化处理，并拒绝会造成目录循环的操作。合法移动通过 MinIO 复制对象到新前缀，再删除旧对象完成。该处理保证了前端目录视图与后端对象名前缀一致。

6.5.2 CSV 与 Markdown 预览边界

CSV 与 Markdown 预览容易受到编码、行列长度和 HTML 渲染安全影响。系统在后端预览服务中将 CSV 组织为表格化 HTML，将 Markdown 转换为独立 HTML 响应，再由前端预览框展示，避免用户上传内容直接进入主页面 DOM。测试结果表明，结构化表格和 Markdown 文本均能形成可读结果。

6.5.3 任务历史状态边界

早期状态字段可能出现 `success` 与 `completed` 混用的问题。当前前端读取任务接口时兼容后端状态，并以任务详情中的 `result_url` 和 `error` 作为结果判断依据。这样即使页面刷新，历史任务仍可从 Redis 状态中恢复，不依赖浏览器临时变量。

6.5.4 PDF 导出文字与坐标边界

PDF 导出测试中重点关注批注框、文本位置和中英文混排。后端转换服务已加入 CJK 字体加载，导出时使用页面尺寸和前端画布尺寸进行坐标映射。导出后的 PDF 能保留文本和矩形框，说明前端批注对象到后端 PDF 坐标之间的映射链路可用。

6.5.5 管理端联动边界

管理端联动测试关注用户端业务增量是否进入实时运行视图。测试方法是在用户端完成上传、转换或 PDF 导出后，立即读取 `/api/v1/tasks/stats`、`/api/v1/logs/stats` 和 `/api/v1/logs/timeline`，再观察管理端第一屏指标与近期事件是否刷新。该测试能够区分实时运行数据和历史分析样本：实时指标来自 FastAPI 聚合接口，会随业务操作变化；历史分析页来自 Spark 输出，只用于较长时间窗口的样本分析。

该边界测试的判定标准不是图表数值越大越好，而是数值来源是否可信、变化是否可解释。若用户上传一个文件，文件列表和日志时间线应出现对应记录；若提交一次转换，任务统计中等待、处理中或成功数量应发生变化；若 PDF 导出成功，结果产物应进入对象存储并形成完成事件。只有这些变化能够被 API 和管理端同时观察到，才能说明系统完成了从用户操作到运行观察的闭环。

图 6.6 展示了用户端业务操作后的管理端指标与日志联动结果。当在用户端上传新文件后，日志统计中的 `file_uploaded` 事件次数及任务事件总数实时增加，证明了联动闭环的可用性。



```
(base) caolei@caoleideMacBook-Air culcloud-platform % curl --no-proxy "*" -sS http://127.0.0.1:8080/api/v1/tasks/stats | python3 -m json.tool
{
  "total": 29,
  "queued": 0,
  "processing": 0,
  "completed": 28,
  "failed": 1
}
(base) caolei@caoleideMacBook-Air culcloud-platform % curl --no-proxy "*" -sS http://127.0.0.1:8080/api/v1/logs/stats | python3 -m json.tool
{
  "total": 34,
  "hours": 24,
  "by_type": {
    "file_downloaded": 12,
    "pdf_reorder_started": 7,
    "pdf_reorder_completed": 7,
    "file_uploaded": 4,
    "conversion_started": 2,
    "conversion_completed": 2
  }
}
(base) caolei@caoleideMacBook-Air culcloud-platform %
```

图 6.6 用户端业务操作与管理端指标联动验证结果

6.6 测试结果分析

6.6.1 核心业务闭环

表 6.7 汇总了本次测试覆盖的核心闭环。每个闭环都要求有明确输入、系统处理、可观察输出和结论，而不是只检查页面是否打开。

表 6.7 核心业务闭环验证结论

闭环	输入	处理与输出	结论
上传与预览	本地文件	MinIO 写入对象, 预览接口识别类型并返回内容地址	对象路径、大小和类型进入用户端视图
格式转换	Markdown/CSV 等文件	Redis 记录任务状态, 转换服务生成 MinIO 产物	当前任务和历史任务均能显示完成状态
PDF 编辑	页面配置与批注	后端重组页面、映射坐标并写入新 PDF	导出结果可打开并保留批注内容
管理观察	用户端操作	Redis 日志、任务统计和健康接口进入管理端图表	用户行为能够进入管理端观察链路
角色隔离	不同角色访问路径	前端路由结合角色接口控制入口	普通用户受限, 管理员可访问后台页面

6.6.2 稳定性与后续改进

当前测试表明系统已具备文件管理、文件预览、格式转换、PDF 编辑和管理端观察的核心能力。仍需在后续版本中加强的部分包括大文件分片上传、多用户权限隔离、高并发转换任务、容器重启后的恢复策略以及 Spark 分析链路的持续增量更新。上述内容属于生产级强化方向, 不影响当前系统作为课程场景下云端资料处理平台的核心功能验收。

因此, 本轮测试结论采用“已满足核心流程、保留生产强化空间”的判定方式, 既确认当前功能可用, 也明确后续优化边界。

从测试覆盖角度看, 当前测试已经覆盖了前端类型、后端路由、角色访问、接口健康和核心业务链路, 但仍需要在后续补充更长时间窗口的运行稳定性测试。例如连续上传多个大文件、并发提交多个转换任务、重启 Redis 后检查任务历史恢复能力、重启 MinIO 后检查对象列表恢复能力、关闭 Gotenberg 后检查转换失败提示。上述测试能够进一步验证系统从可运行原型向长期运行平台演进时的可靠性。

6.7 本章小结

本章从测试环境、功能测试、命令行验证、问题处理和结果分析几个方面验证了 CulCloud。测试结果说明，系统能够完成上传、预览、转换、PDF 页面导出、状态记录、角色隔离和管理端观察等关键流程，达到了系统设计目标。

第 7 章 总结与展望

本章对 CulCloud 云盘与大数据分析平台的设计与实现工作进行总结，并说明系统当前不足和后续改进方向。

7.1 总结

7.1.1 系统功能成果

本文围绕 CulCloud 云盘与大数据分析平台完成了系统设计、前端实现、文件处理、PDF 编辑室、任务监控和文档工程规划。项目实现了真实首页指标、拖拽上传、文件夹管理、文件分类、专门文件预览页、PDF canvas 批注、页面整理、打印导出和管理端遥测分析。

7.1.2 工程实践成果

项目将大数据分析、云存储、任务队列、文件转换和前端可视化结合起来，能够支撑办公文档处理、教学资料整理和实验文件归档等多种应用场景。系统中的上传、转换、预览和失败记录也为后续 Spark 聚合分析提供了数据来源。

7.2 不足之处

7.2.1 PDF 批注写回不足

当前 PDF 工作台的批注功能主要在前端 Web 端的 Canvas 视图层及打印导出中生效，尚未实现将用户的绘图 and 文本批注直接持久化写回 PDF 底层文件结构中。其核心技术成因在于：一是 PDF 规范采用笛卡尔坐标系（原点在左下角），而前端 Canvas 容器采用逻辑像素坐标系（原点在左上角），在页面缩放与旋转需要进行复杂的仿射变换矩阵计算；二是系统缺乏中文字体的子集化嵌入机制，强行写回会导致不同终端查看时因缺少中文字体而无法正确显示字符；三是涉及 PDF 增量更新与注解对象（Annotation Object）字典树的序列化写回算法尚未在后端服务中实现，难以保证 PDF 文件的二进制结构合规与历史版本兼容。

7.2.2 Office 深度预览不足

由于现代浏览器无法原生解析并高保真呈现微软 Office 办公套件格式，系统在处理此类文件时，必须采用静默调用 Gotenberg 后端服务将其转换为 PDF 并写入 Redis 缓存库的过渡方案。这一方案深度依赖转换引擎的单点可用性。在 Gotenberg 容器发生宕机、转换任务队列发生阻塞或面对超大尺寸文件时，系统缺乏成熟的降级重试与故障平滑恢复机制，极易导致前端预览界面无限期处于加载状态，进而削弱了预览链路在极端高负载下的可用性。

7.2.3 权限与协作能力不足

网盘分享、多人协作、权限体系和生产级鉴权尚未纳入当前实现范围。当前系统更适合单用户资料整理和轻量级文档处理场景，距离多人协同网盘仍有差距。

7.3 展望

7.3.1 增强 PDF 编辑能力

后续计划引入 Fabric.js 构建更具交互性的富文本批注图层，替代目前基于简单 HTML5 Canvas 的手写体涂鸦。在此基础上，借助 pdf-lib 库在 Node.js 服务端重构批注的合规写回引擎，确保用户的几何标注、高亮文本与中文批注能够以标准注释对象的形式注入 PDF 二进制结构中。同时，优化多终端的批注缩放自适应算法，提供批注节点的增删改查和版本历史回溯服务。

7.3.2 完善多格式文件处理

后续可继续完善 Office、音频、视频、压缩包和大文件的上传、预览与转换策略，并通过更完整的白盒测试保证转换链路稳定。

7.3.3 扩展数据分析指标

未来将在数据分析大屏中引入更深度的 Spark 离线计算与实时遥测指标，深入评估接口每秒查询率（QPS）和数据质量瓶颈。通过精细化采集文件传输耗时、转换队列排队时长以及不同格式文件解析的出错率，运用 Spark 分析多维度的性能瓶颈点，为底层 Gotenberg 转换实例的水平扩容以及 Redis 缓存淘汰策略 of 动态调整提供量化决策支撑。

致 谢

在本项目完成之际，首先要感谢李亦飞老师在课程学习与项目实践中的悉心指导和宝贵建议。李老师深入浅出的讲解使我们对大数据技术有了系统的认识，为项目的顺利开展奠定了坚实的理论基础。

感谢课程小组各位成员的共同努力和密切协作，大家在项目开发过程中相互帮助、共同进步，顺利完成了系统的设计与实现。

感谢常州工学院为我们提供了良好的学习环境和实验条件，使我们能够将理论知识转化为实践成果。

最后，感谢所有在学习和生活中给予关心和支持的家人和朋友。

参考文献

- [1] 全国人民代表大会常务委员会. 中华人民共和国网络安全法[EB/OL]. 2016. <https://flk.npc.gov.cn/detail2.html?MmM5MDlmZGQ2NzhiZjE3OTAxNjc4YmY4Mjc2ZjA5M2Q%3D>.
- [2] 全国人民代表大会常务委员会. 中华人民共和国数据安全法[EB/OL]. 2021. <https://flk.npc.gov.cn/detail2.html?ZmY4MDgxODE3OWY1ZTA4MDAxNzlmODg1YzdlNzAzOTI>.
- [3] 全国人民代表大会常务委员会. 中华人民共和国个人信息保护法[EB/OL]. 2021. <https://flk.npc.gov.cn/detail2.html?ZmY4MDgxODE3YjY0NzJhMzAxN2I2NTZjYzIwNDAwNDQ>.
- [4] 国务院. 促进大数据发展行动纲要[EB/OL]. 2015. https://www.gov.cn/zhengce/content/2015-09/05/content_10137.htm.
- [5] Fortune Business Insights. Cloud storage market size, share and industry analysis, 2026–2034 [EB/OL]. 2026. <https://www.fortunebusinessinsights.com/cloud-storage-market-102773>.
- [6] DataIntel. Global file converter software market report size & share[EB/OL]. 2025. <https://dataintel.com/report/global-file-converter-software-market>.
- [7] DataIntel. Global pdf editor market report size & share[EB/OL]. 2026. <https://dataintel.com/report/global-pdf-editor-market>.
- [8] Google. Google drive help[EB/OL]. 2026. <https://support.google.com/drive/>.
- [9] Dropbox. Dropbox help center[EB/OL]. 2026. <https://help.dropbox.com/>.
- [10] Meta Open Source. React documentation[EB/OL]. 2026. <https://react.dev/>.
- [11] MDN Web Docs. Canvas api[EB/OL]. 2026. https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API.
- [12] FastAPI. Fastapi documentation[EB/OL]. 2026. <https://fastapi.tiangolo.com/>.
- [13] Pallets Projects. Flask documentation[EB/OL]. 2026. <https://flask.palletsprojects.com/>.
- [14] MinIO. Minio object store documentation[EB/OL]. 2026. <https://min.io/docs/minio/linux/index.html>.
- [15] Gotenberg. Gotenberg documentation[EB/OL]. 2026. <https://gotenberg.dev/docs>.
- [16] Mozilla. Pdf.js[EB/OL]. 2026. <https://mozilla.github.io/pdf.js/>.
- [17] PM2. Pm2 documentation[EB/OL]. 2026. <https://pm2.keymetrics.io/docs/usage/quick-start/>.
- [18] Docker. Docker compose documentation[EB/OL]. 2026. <https://docs.docker.com/compose/>.
- [19] Shvachko K, Kuang H, Radia S, et al. The hadoop distributed file system[J]. Proceedings of the IEEE Symposium on Mass Storage Systems and Technologies, 2010: 1-10.
- [20] Zaharia M, Xin R S, Wendell P, et al. Apache spark: A unified engine for big data processing [J]. Communications of the ACM, 2016, 59(11): 56-65.